



Estilos y Patrones I

IIC2173 - Arquitectura de sistemas de software

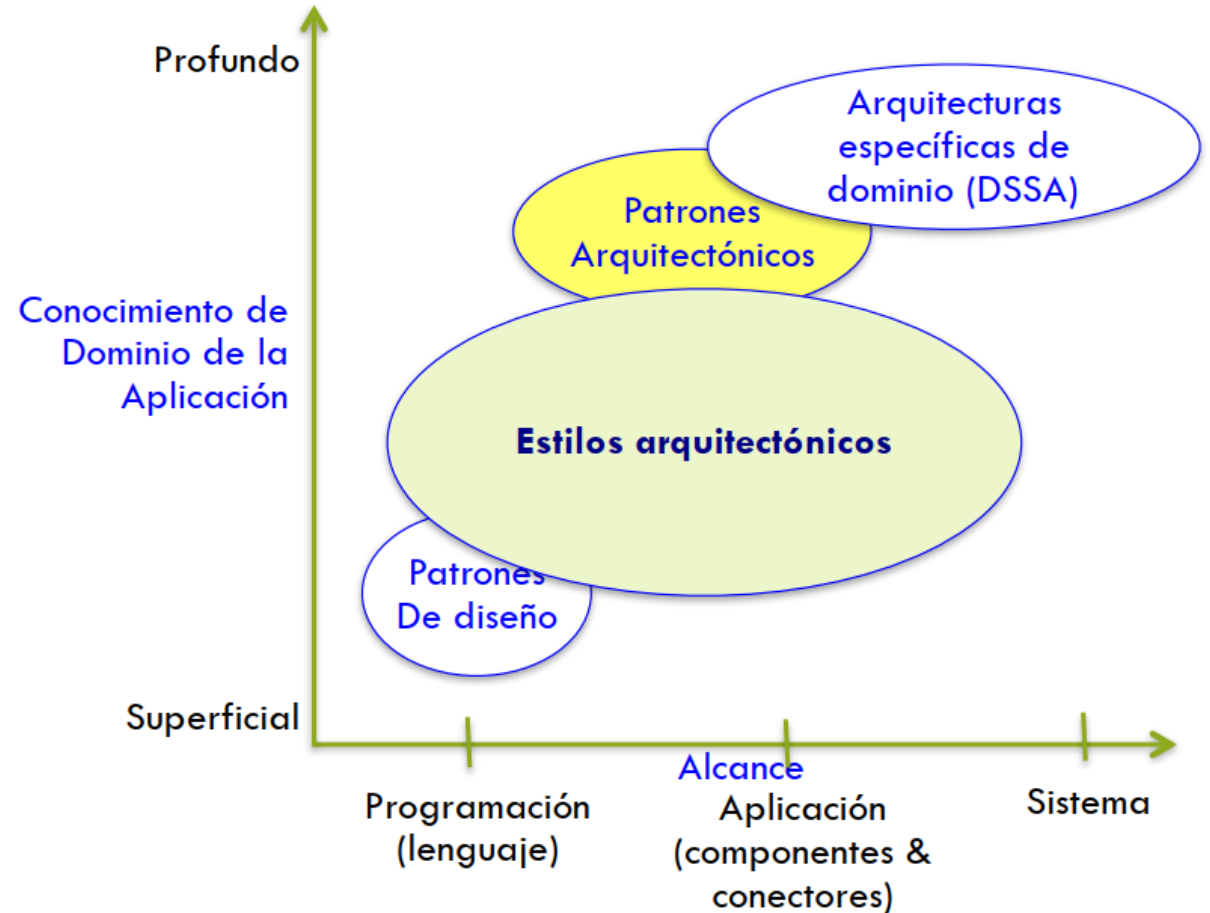
Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

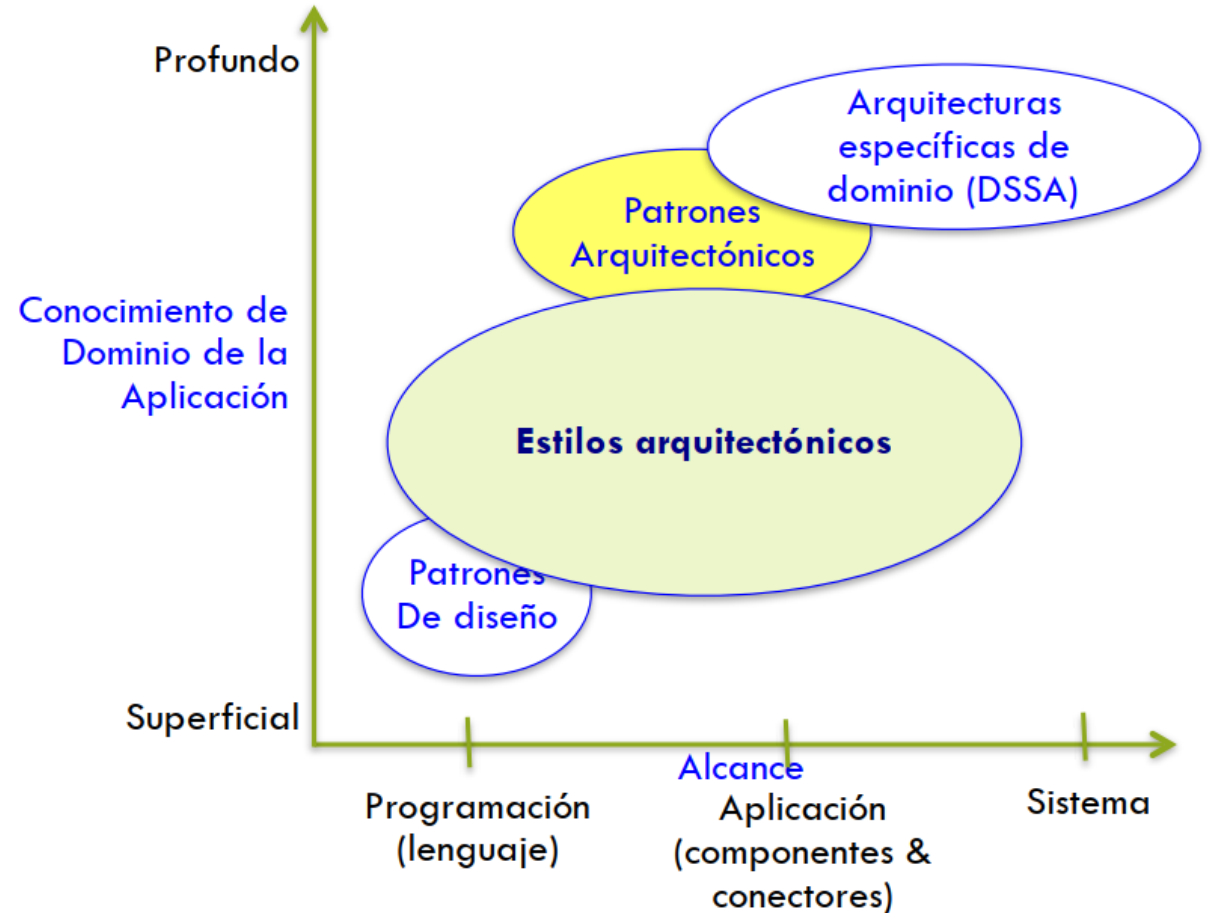
Estilos arquitectónicos

- Colección nombrada (con nombre) de decisiones de diseño arquitectónico que:
 - Aplican a un **contexto de desarrollo dado**
 - Restringen las decisiones de diseño que **son específicas a un sistema particular** dentro de ese contexto
- Promueven ciertos atributos de calidad
- Gobiernan
 - Código
 - Componentes
 - Requisitos
 - Decisiones



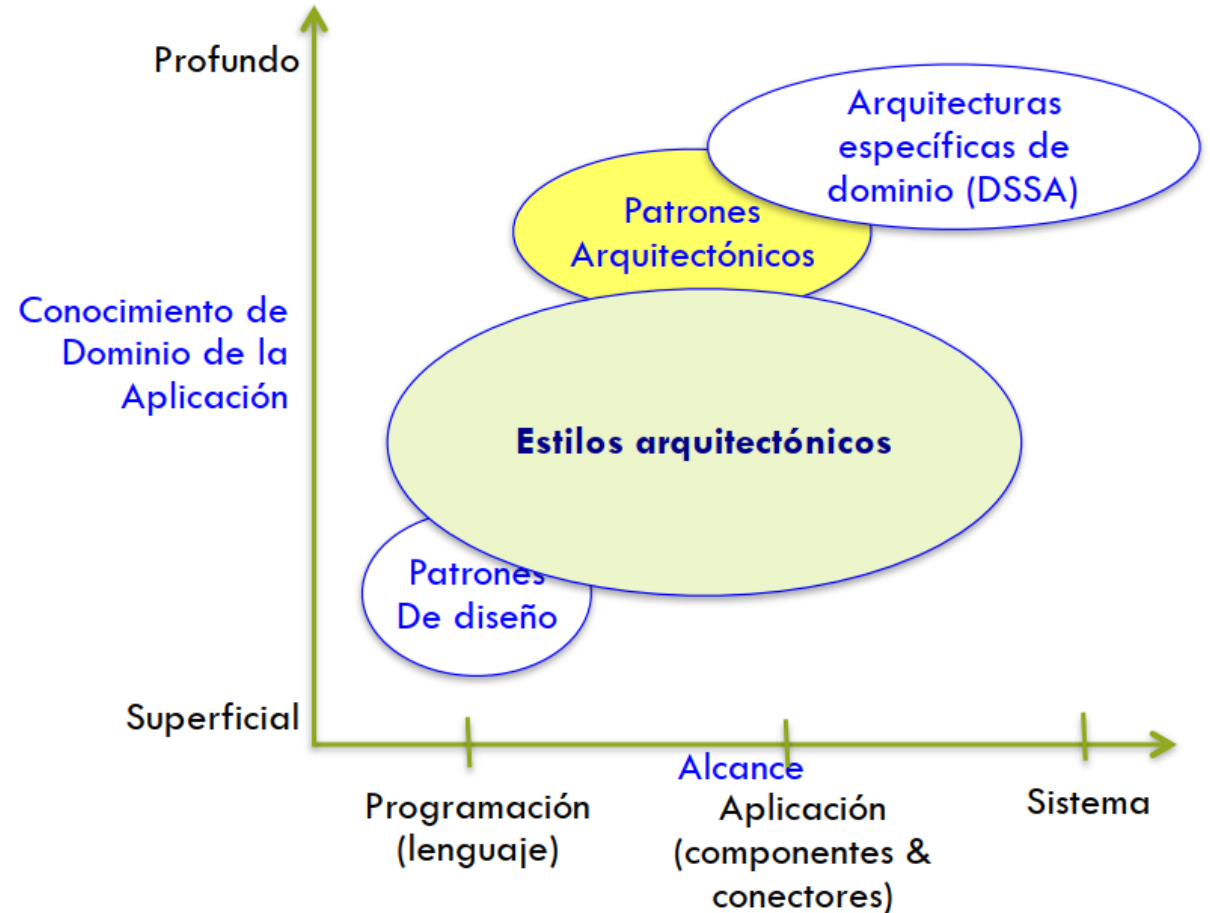
Patrones de Arquitectura

- Colección nombrada de decisiones de diseño arquitectónico que:
 - Aplican **a un problema de diseño recurrente**
 - Se parametriza **según los contextos de desarrollo** en que aparece el problema



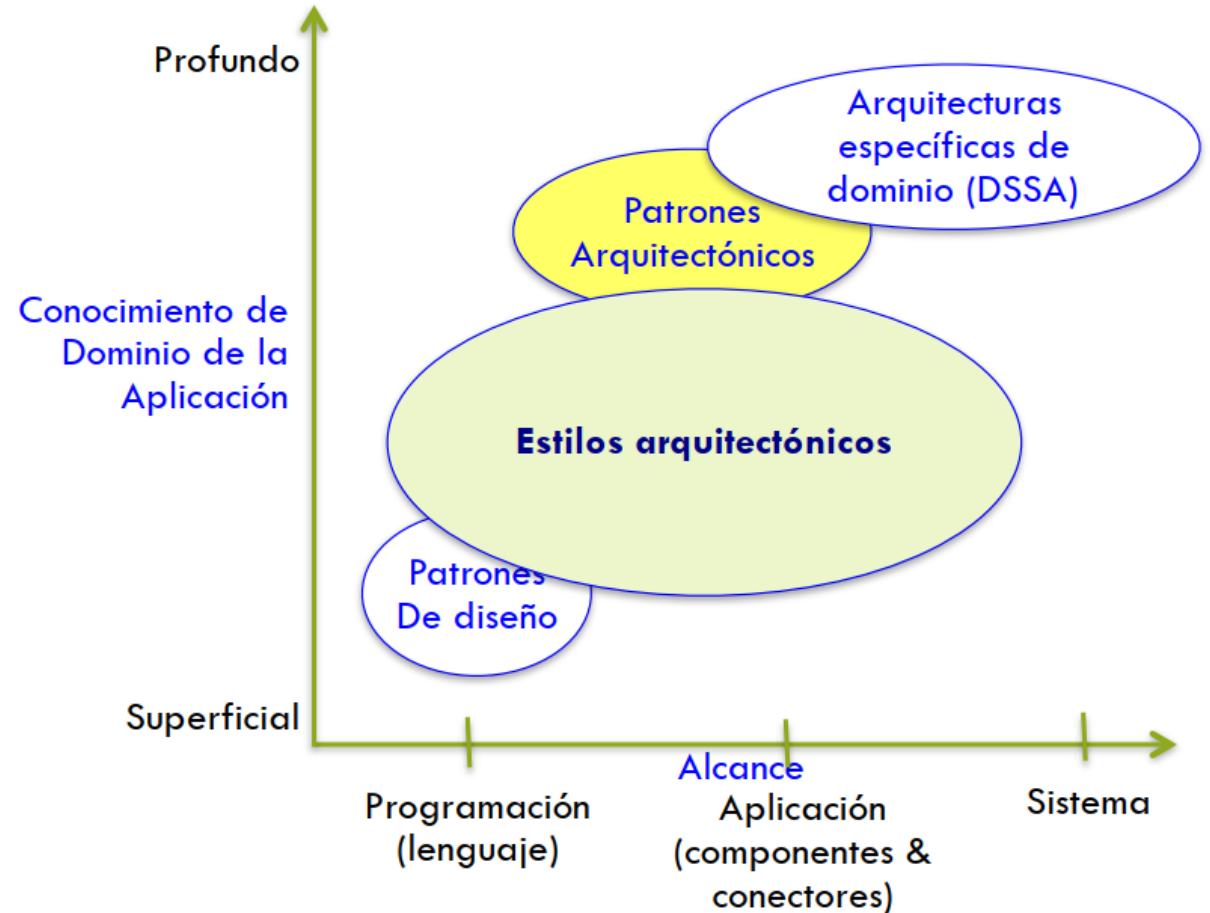
Diferencias entre estilo y patrón

- **Alcance:** Problema de diseño vs Contexto de desarrollo
- **Abstracción:** Fragmentos arquitectónicos parametrizados vs Requiere interpretación humana
- **Relación:** Un patrón puede aplicarse a sistemas que siguen varios estilos, un sistema que sigue un estilo puede usar varios patrones



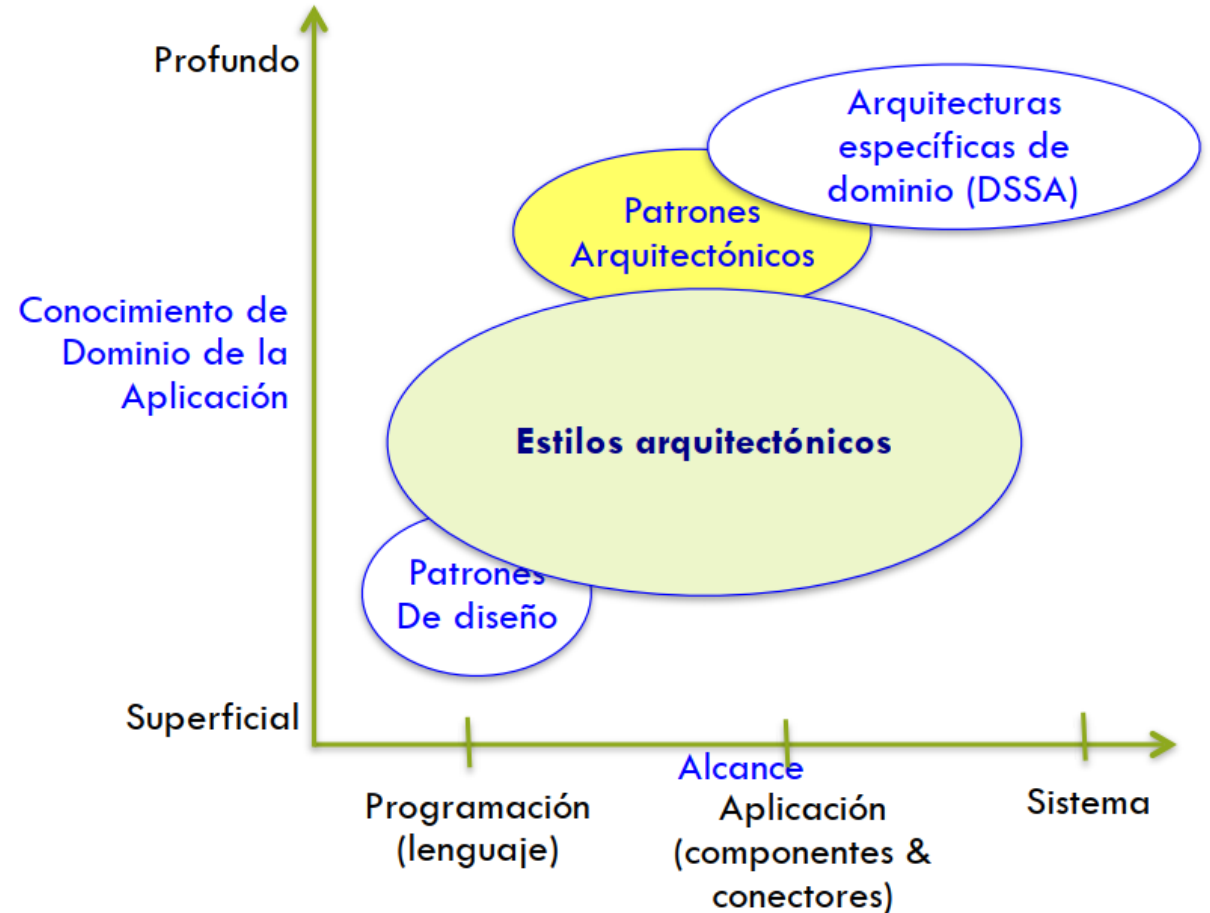
Ventajas de estilos y patrones

- Reuso de diseños probados
 - Soluciones bien entendidas aplicadas a nuevos problemas
- Genera soluciones más maduras, establecidas
- Lenguaje compartido mejora comunicación
 - Ej: "Cliente-Servidor" ya indica cómo es el sistema
- DRY, se apoya en convenciones



Ventajas de estilos y patrones

- Promueven **interoperabilidad**
 - Debido a la estandarización del estilo y su mayor comprensión
- Análisis específico al estilo
 - Permite **restringir el espacio de diseño y justificar decisiones** más fácilmente
- **Visualizaciones**
 - Figuras específicas al estilo calzan con el modelo mental "ingenieril"





Qué queremos evitar:
Big ball of mud

Monolito: una primera arquitectura

- Sistema contenido en una sola quanta
- Un bloque de software, una base de datos
- Alta cohesión, acoplamiento estático, aunque aprecia modularidad
- Implicancias dependientes del estilo, no es inherentemente malo (debatible)



Estilos comunes

1. Influenciados por el lenguaje

1.1 Main y sub-rutinas

1.2 Orientado al objeto

2. En capas

2.1 Máquinas virtuales

2.2 Cliente - Servidor

3. Microkernel

4. Flujo de datos

4.1 Por Lotes, secuencial

4.2 Tubería y Filtro (Pipe & Filter)

5. Memoria compartida

5.1 Pizarra

5.2 Basado en reglas

6. Intérprete

6.1 Intérprete

6.2 Código móvil

7. Invocación implícita

7.1 Basado en eventos

7.2 Publisher – Subscriber

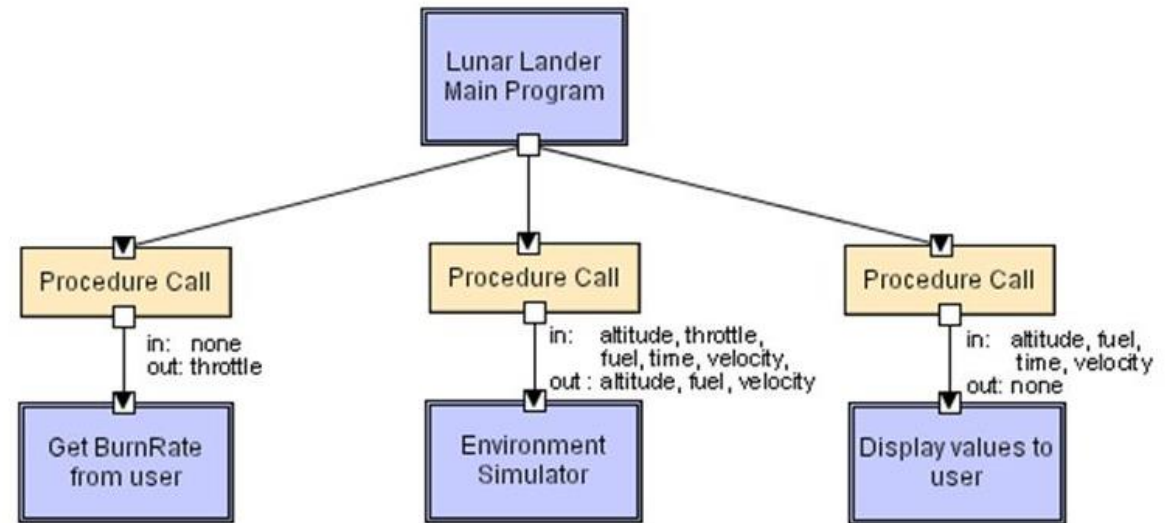
8. P2P (Peer to Peer)

9. Derivados

C2, Corba

Main y subrutinas: Descripción

- Arquitectura históricamente relevante
- Subrutinas individuales y encapsuladas
- *Main* “cede” el control a estas subrutinas y determina el orden
- Alto acoplamiento y bloqueante



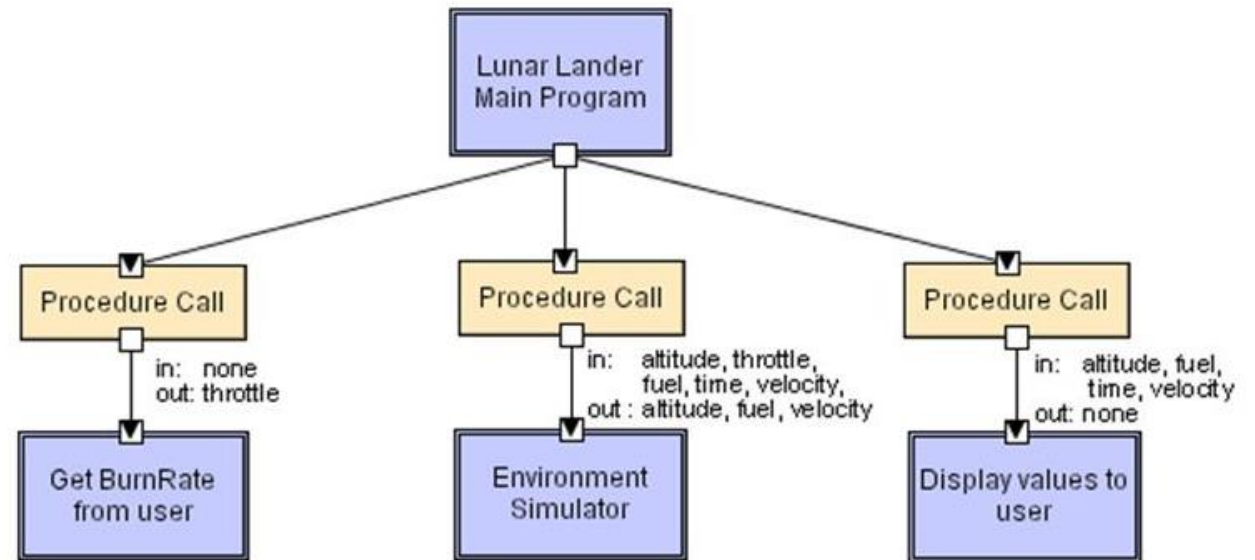
Main y subrutinas: Ejemplo Lunar Lander

- Solo se realiza una acción a la vez
- Otros ejemplos
 - Tetris
 - Sistemas embebidos
 - MS-DOS
 - Pac-Man



Main y subrutinas: Datos y topología

- Datos:
 - Parámetros de llamada de función
- Topología:
 - Centralizado en rutina *main*: ida y vuelta



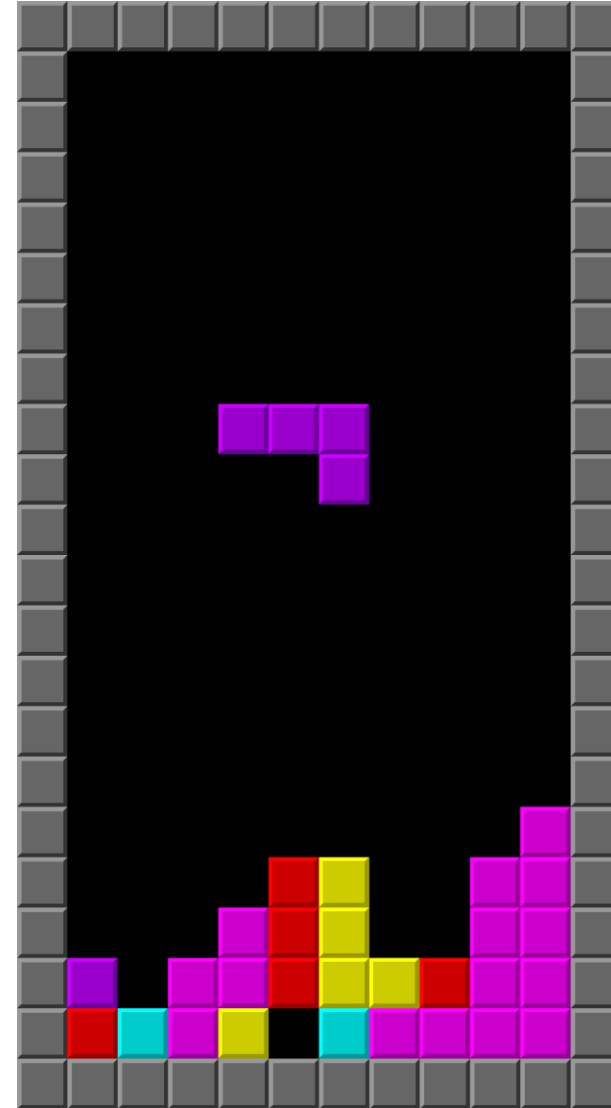
Main y subrutinas: Cualidades

- Modularidad: Si se respeta la interfaz, puedes modificar la subrutina
- Nulo costo de comunicación: *calls* casi instantáneas
- Varios desarrolladores pueden trabajar en diversas partes de la aplicación (pero no muchos)



Main y subrutinas: Advertencias

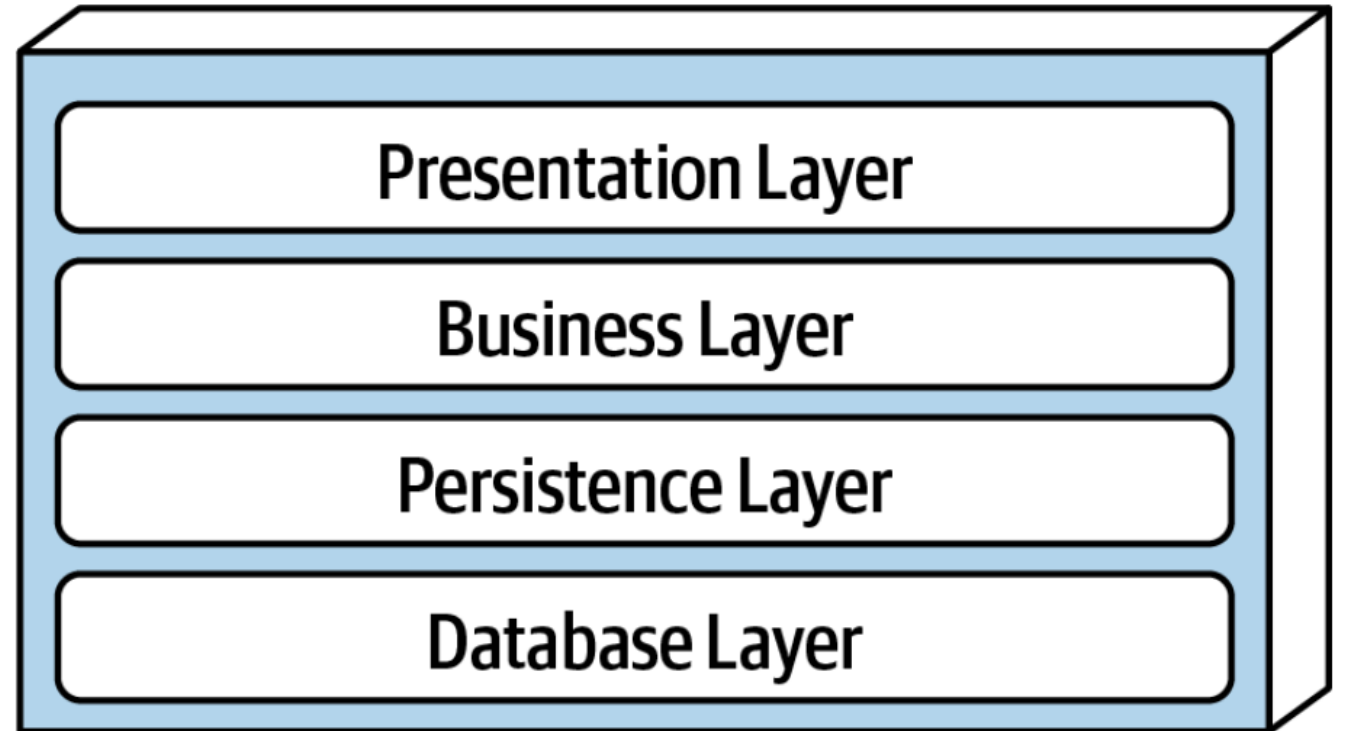
- Pobre extensibilidad y escalabilidad
- Incompatible con estructuras de datos más complejas
- Limitaciones de performance
- Posibilidad de bloqueo



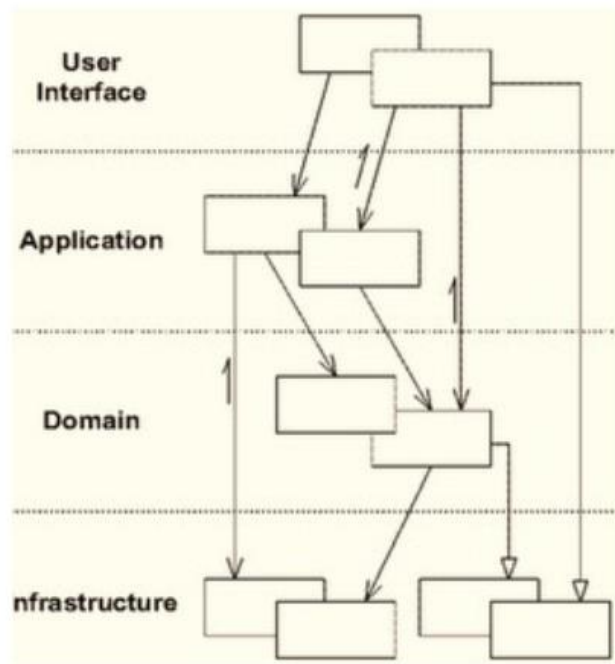
En capas: Descripción

Organización en **capas lógicas horizontales con roles específicos** en la aplicación

- Estilo muy común
 - Suele salir de una división técnica
- Capas fuertemente ordenadas y cerradas
 - Cerrado: Solo puedes interactuar con capas adyacentes
 - Abierto: Puedes interactuar entre capas
- Cada capa encapsula funcionalidad



En capas: Ejemplos



Modelo abierto

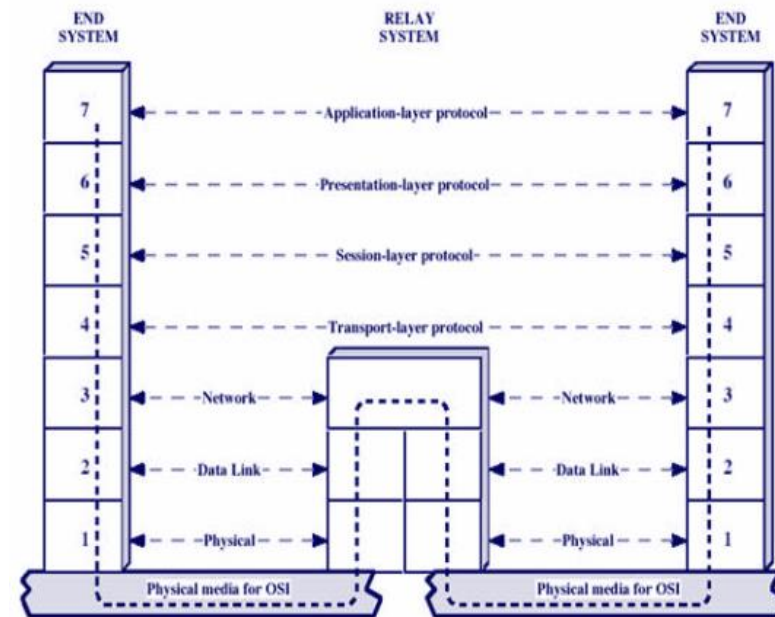


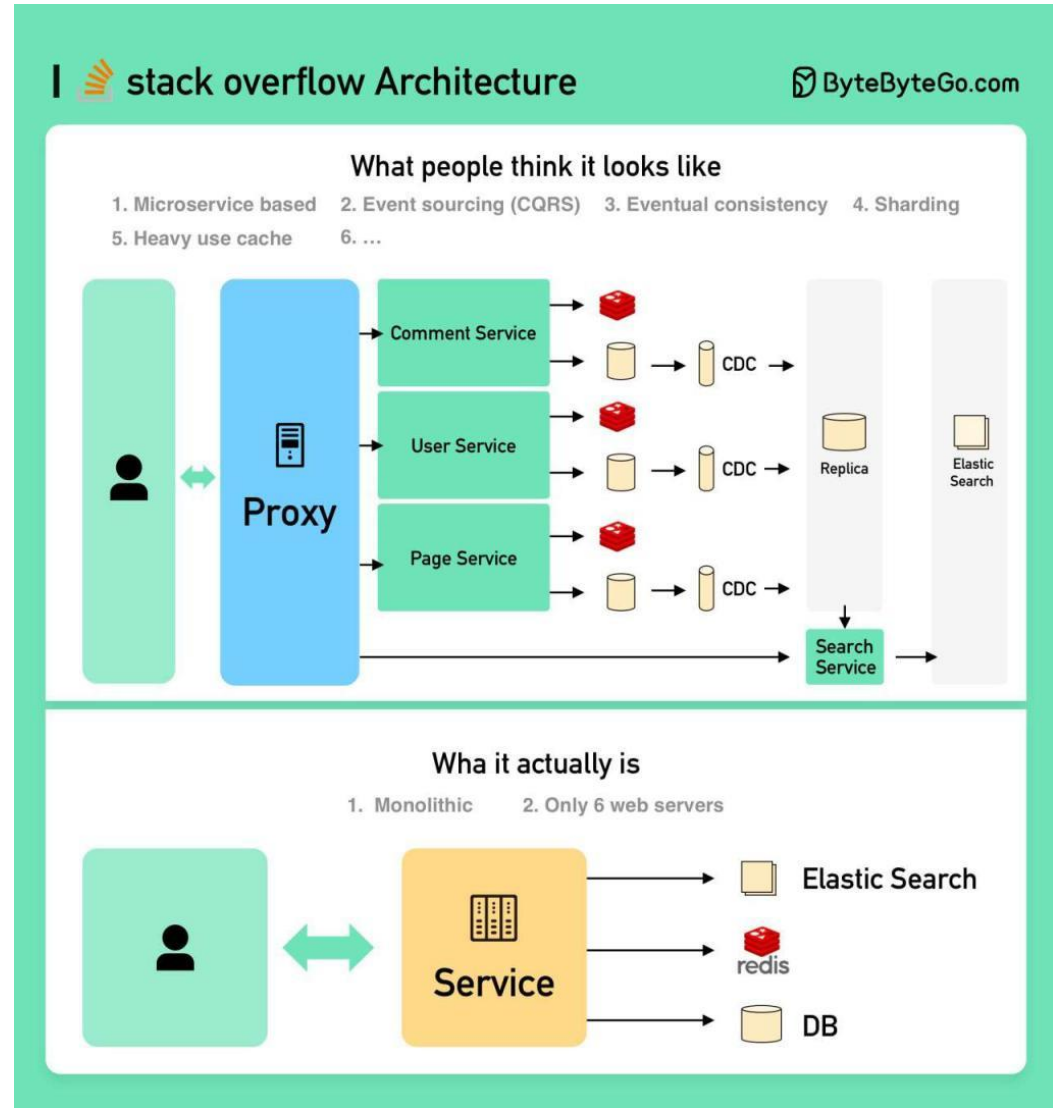
Figure 2.11 The Use of a Relay

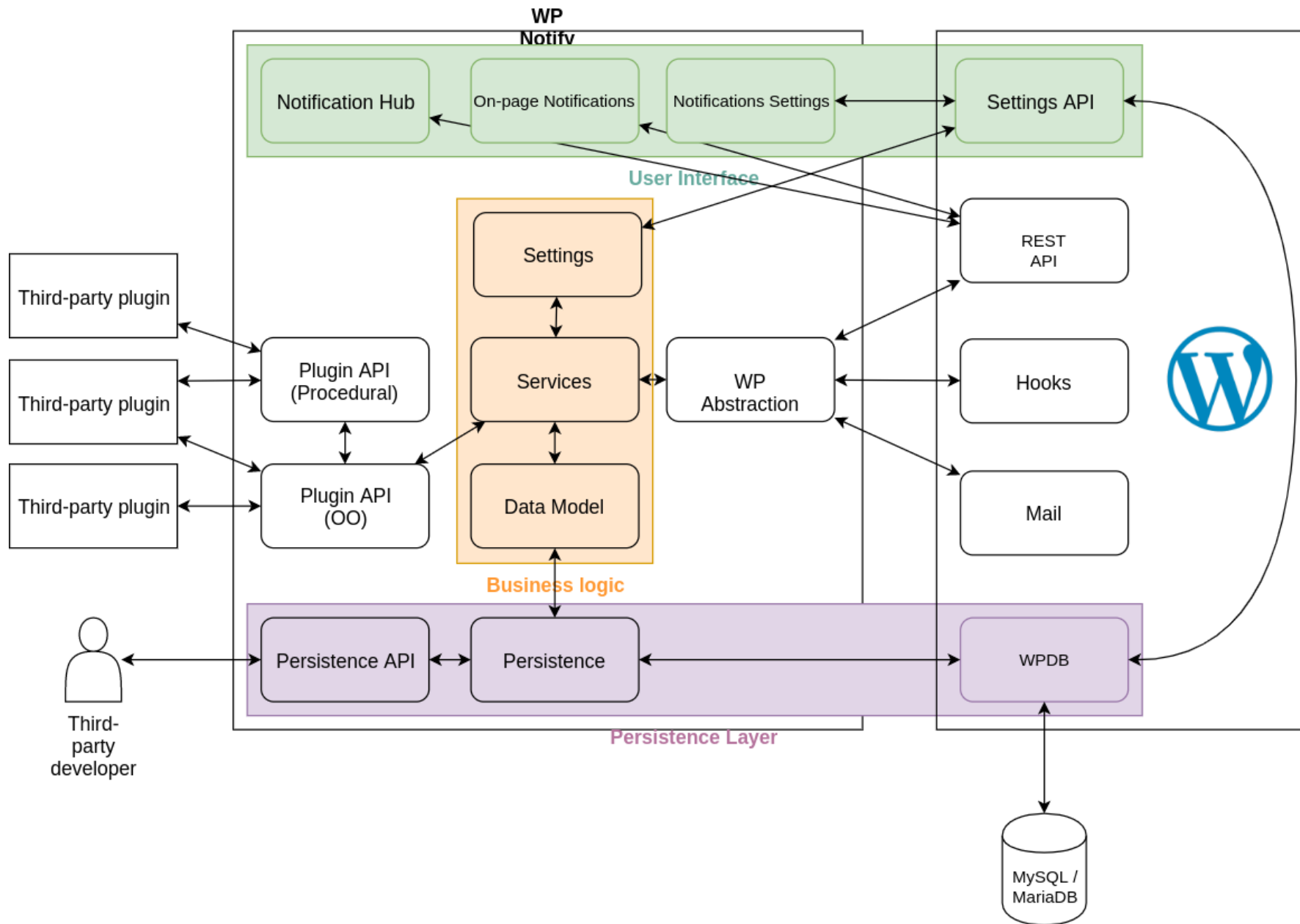
**Modelo cerrado
OSI**

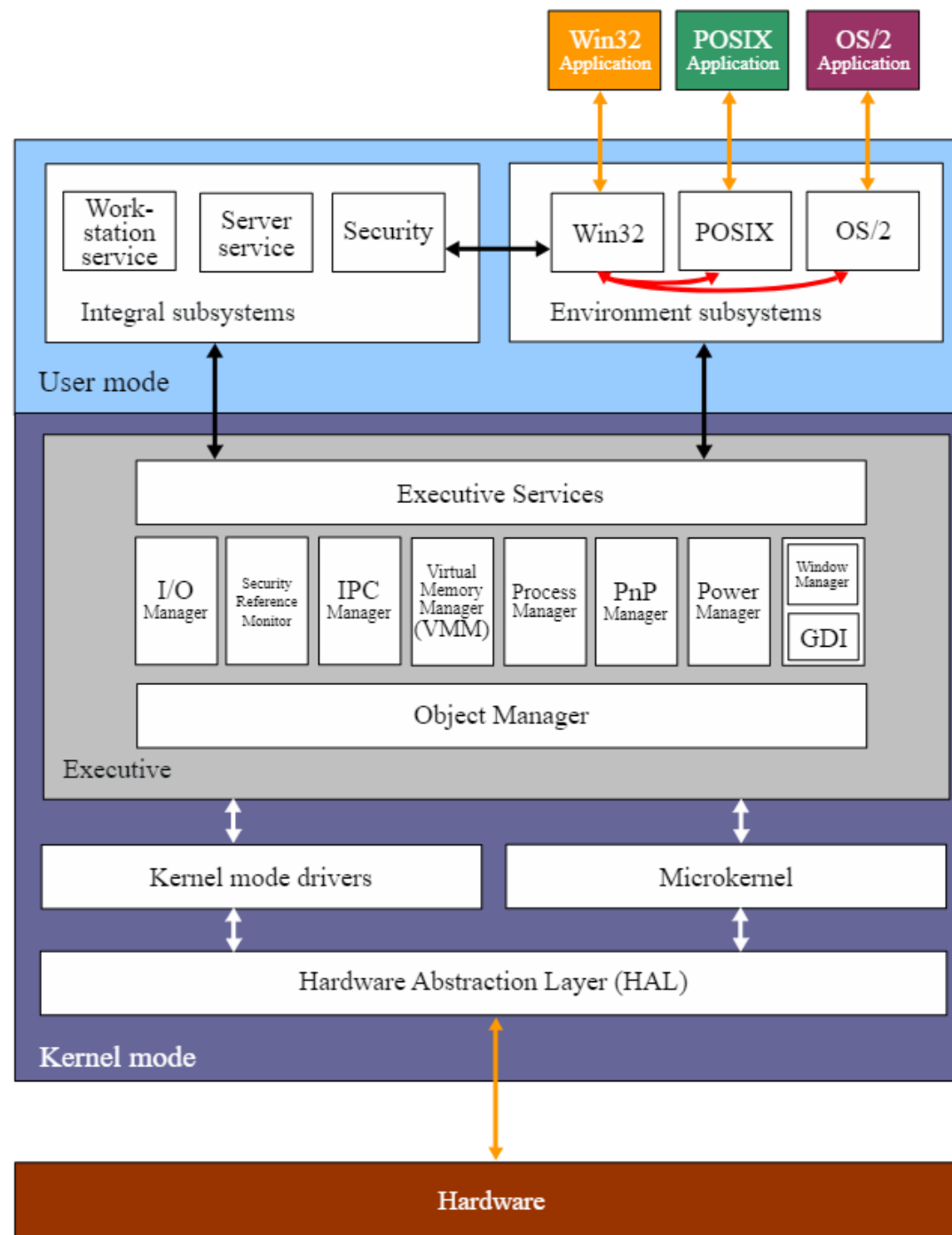
En capas: Ejemplos

<https://stackoverflow.com/performance>

- Rails
- Máquinas virtuales
- Servicios web
- E-commerce
- Ciertos Kernels de sistema
 - Linux, Windows







Windows - NT Kernel

Linux kernel SCI (System Call Interface)

I/O subsystem

Linux kernel

Virtual File System

Terminals

Sockets

File systems

Netfilter / Nftables

Network
protocols

Generic
block layer

Linux kernel
Packet Scheduler

Linux kernel
I/O Scheduler

Character
device
drivers

Network
device
drivers

Block
device
drivers

Line
discipline

Memory management subsystem

Virtual
memory

Paging
page
replacement

Page
cache

Process management subsystem

Signal
handling

process/thread
creation &
termination

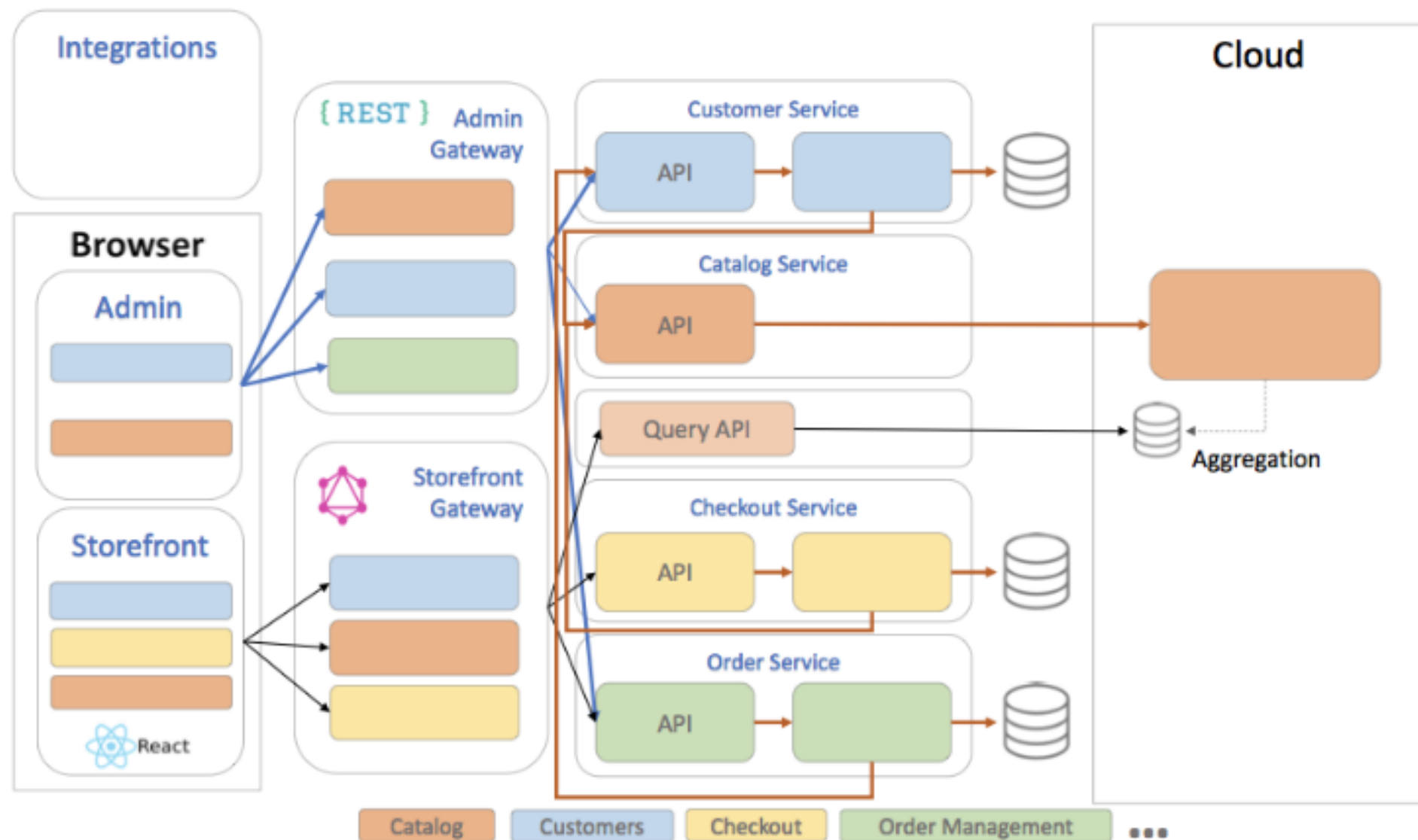
Linux kernel
Process
Scheduler

IRQs

Dispatcher

Desired state

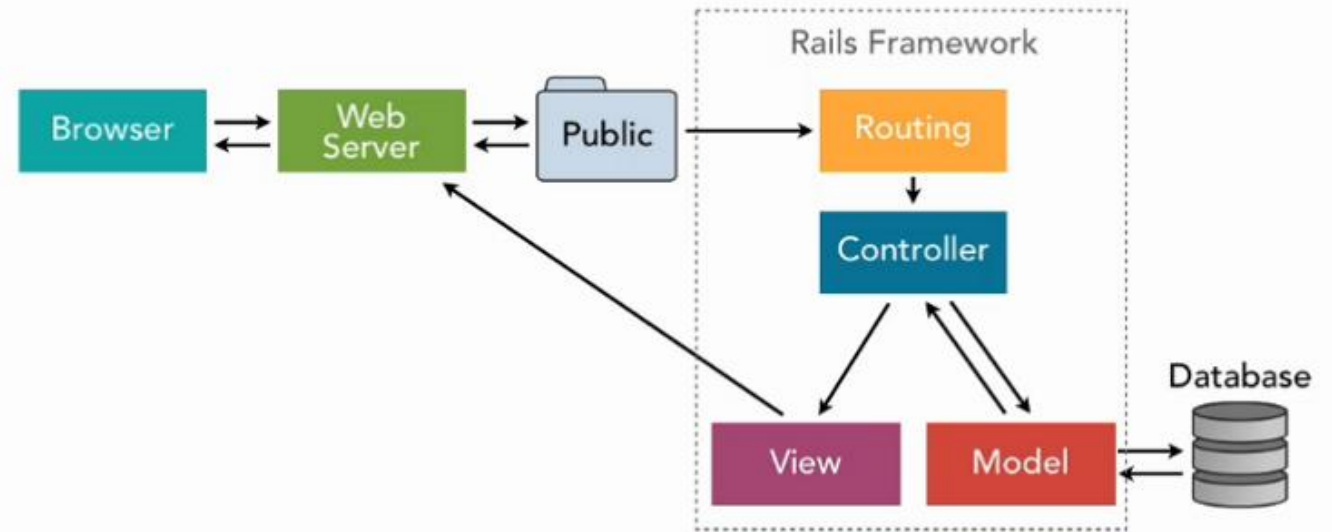
The application consists of technically independent, separately and/or jointly deployable\scalable\replaceable\maintainable parts - services.

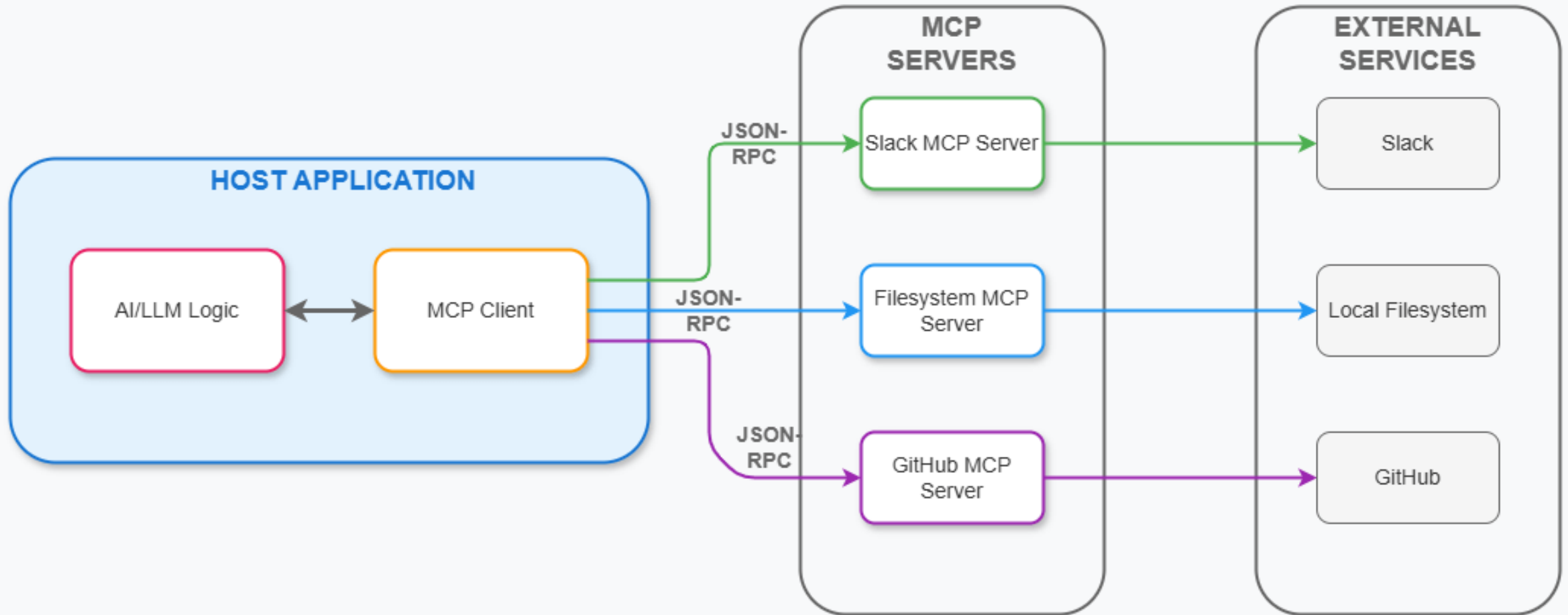


En capas: Datos y topología

- Datos: Parámetros y paquetes
 - Bien específicos, concisos
 - Interfaz demanda estructura específica
- Topología
 - Lineal: datos se mueven en direcciones predecibles

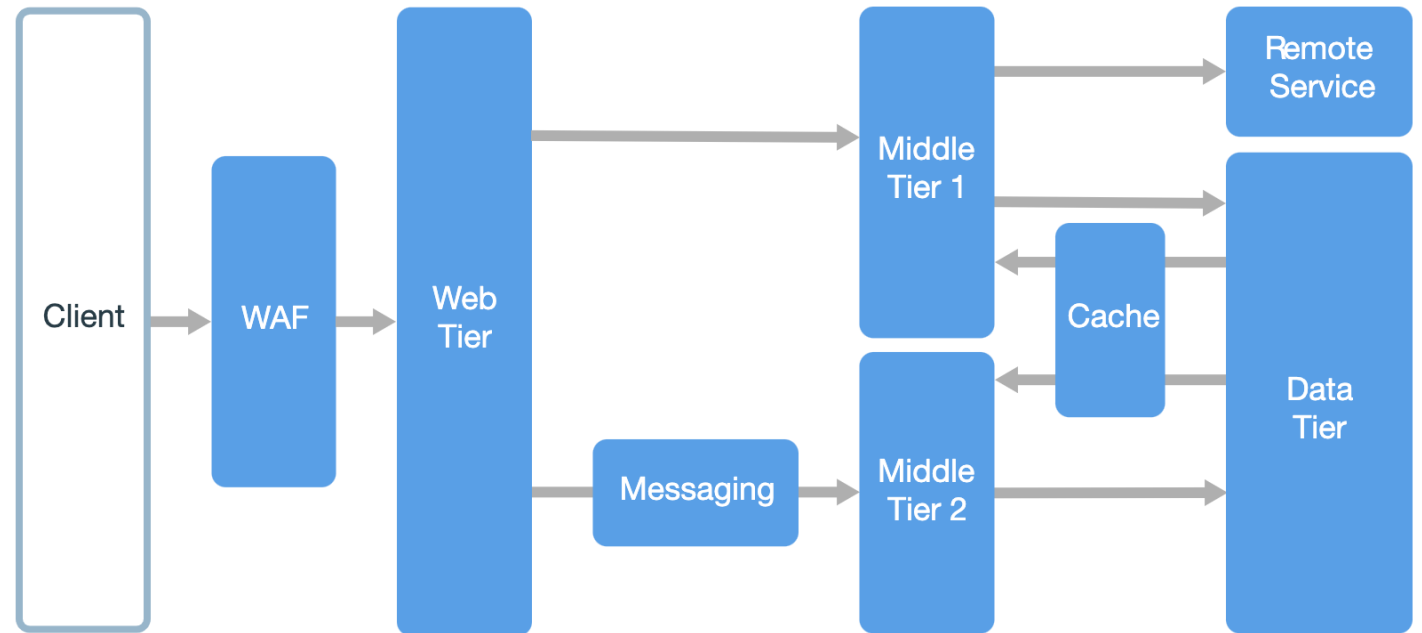
Rails architecture





En capas: Pros

- Estilo muy conocido
 - Fácil de diseñar y entender
 - Suele reflejar estructuras de negocio
- Aumenta niveles de abstracción
 - Dependencias muy claras
 - Permite cambiar cosas en una capa solo afectando capas adyacentes
- Interfaces precisas
- Económico y rápido de crear



Por capas: Cons

- Pobre desempeño a gran escala
 - Bajo paralelismo comparativamente
- Un exceso de capas reduce performance
- Algunos diseños pueden bloquearse con respuestas
- No es universalmente aplicable
- Difícil evolucionar la arquitectura debido a la rigidez
- Poco tolerante a fallos
- Pueden originarse *tiers* anémicos



Variante de capas: Máquina virtual

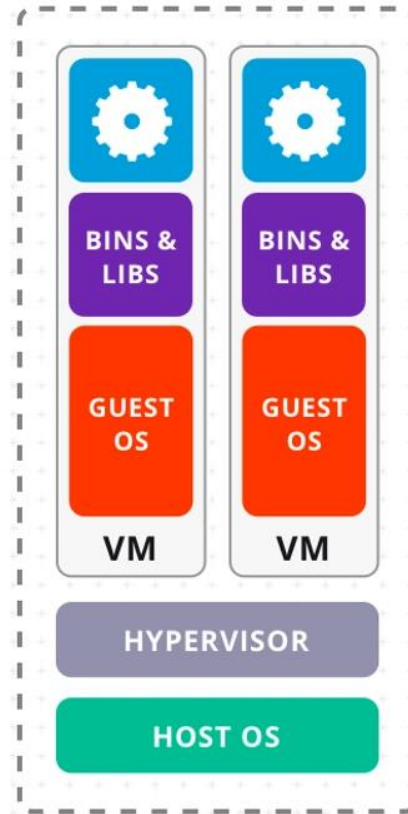
Capacidad de **proveer de recursos a múltiples sistemas/subsistemas completos en una máquina física**

- Servicio “hacia arriba”
- Gran sustento de la infraestructura de muchos de los sistemas de hoy, incluyendo *cloud*
- Hoy sustentado por muchos complejos de hardware

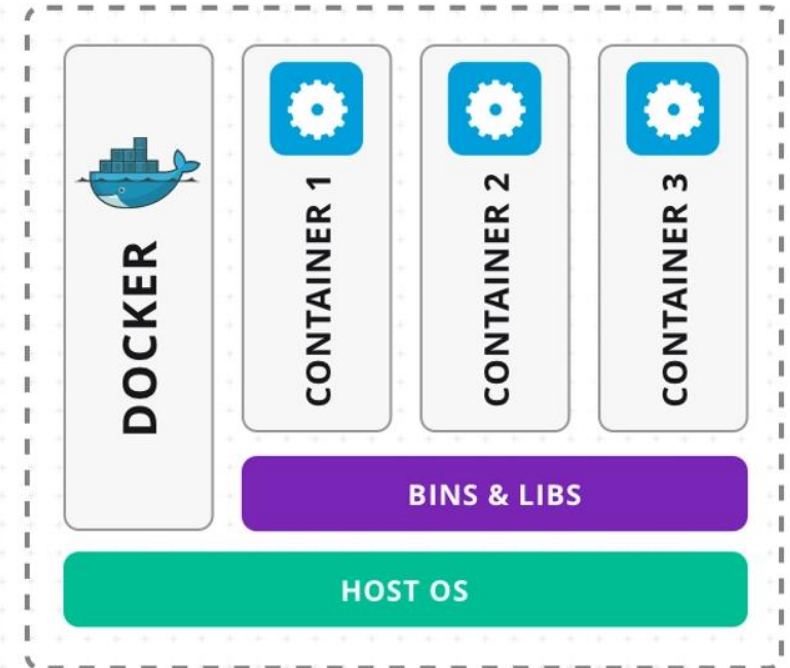


Máquina virtual: Datos y topología

- Componentes:
 - Programas (subcomponentes) a ser ejecutados
 - Motor de ejecución
 - Estado (datos) de los programas
 - Estado (datos) internos del motor
- Conectores: Llamada a procedimientos
- Topología: Capas cerradas o Grafo Acíclico Dirigido



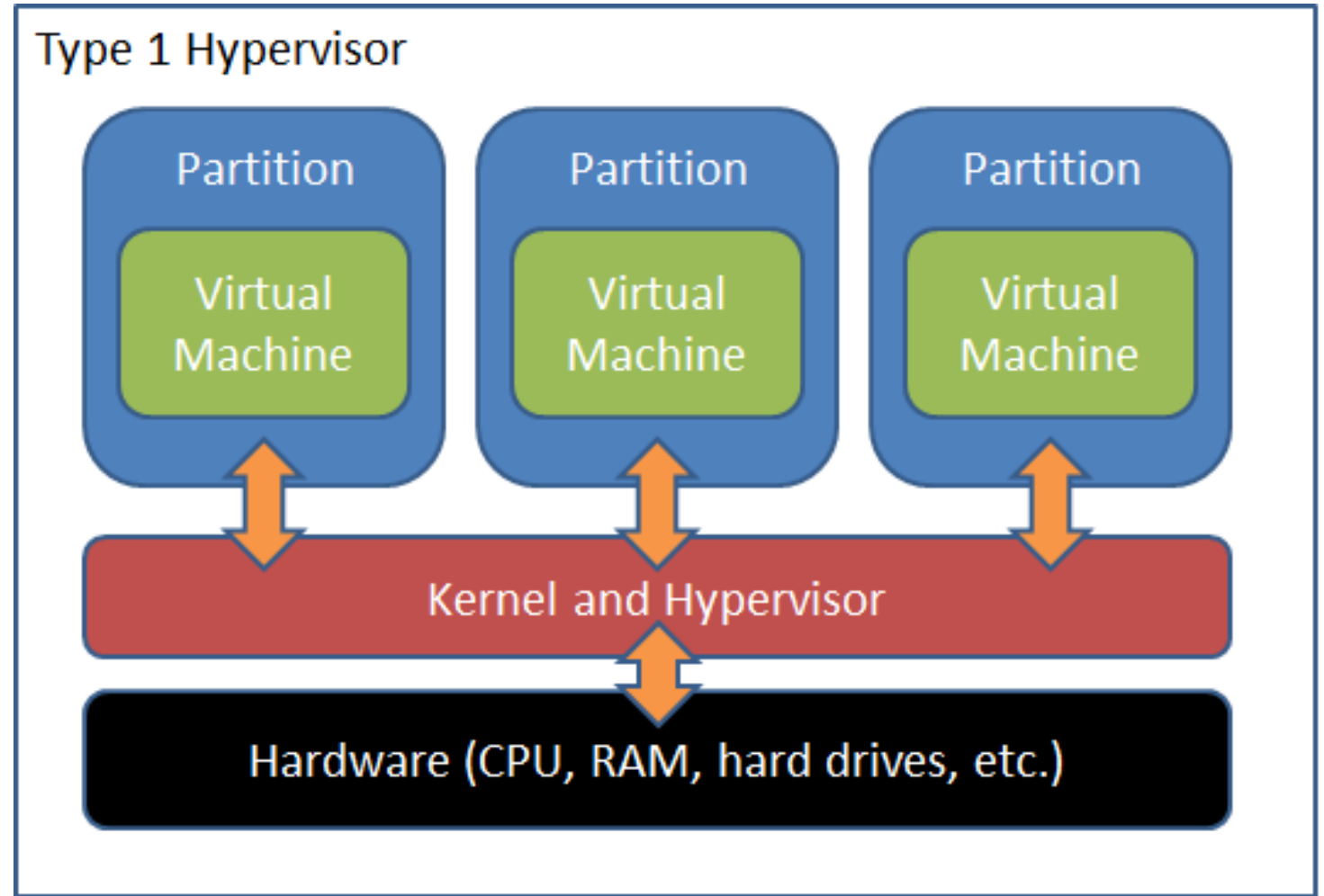
SERVER WITH
VIRTUAL MACHINES

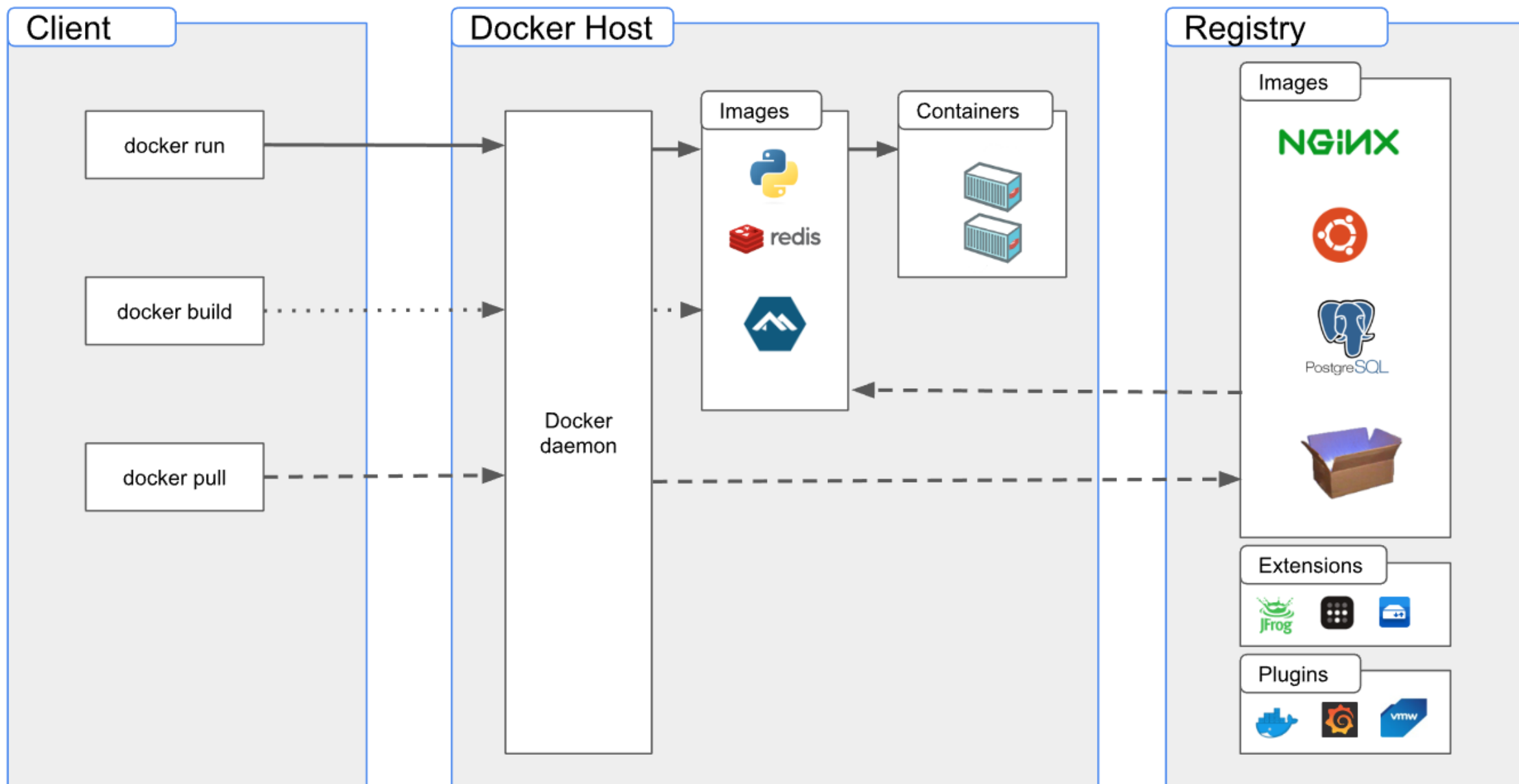


SERVER WITH
DOCKER CONTAINERS

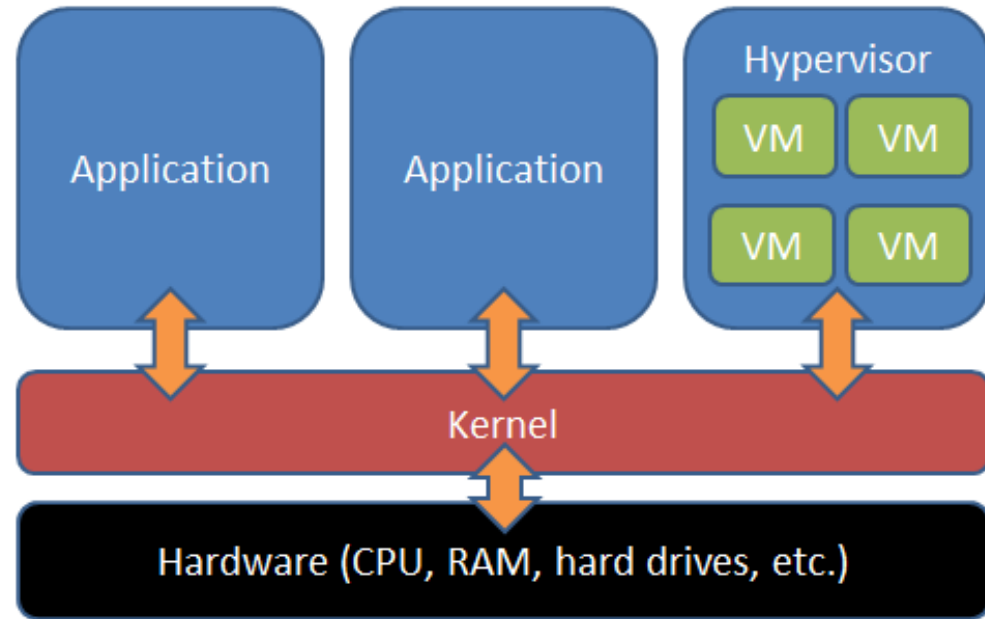
Máquina virtual: Ejemplos

- Hipervisores
 - Tipo 1
 - Xen
 - Hyper-V
 - Tipo 2
 - Vmware
 - VirtualBox
- Containers
 - Docker
 - Runc

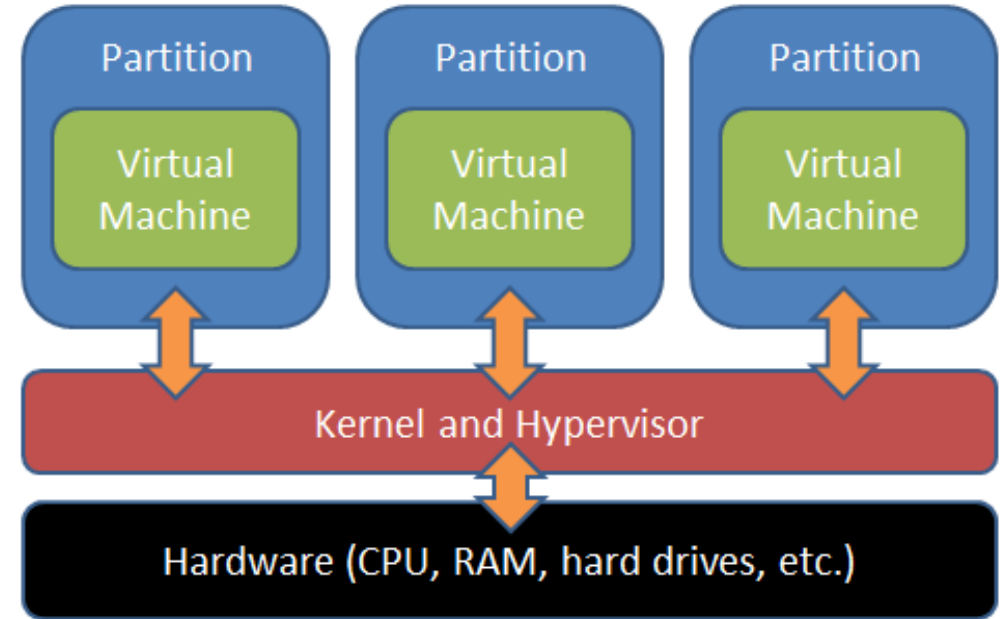




Type 2 Hypervisor

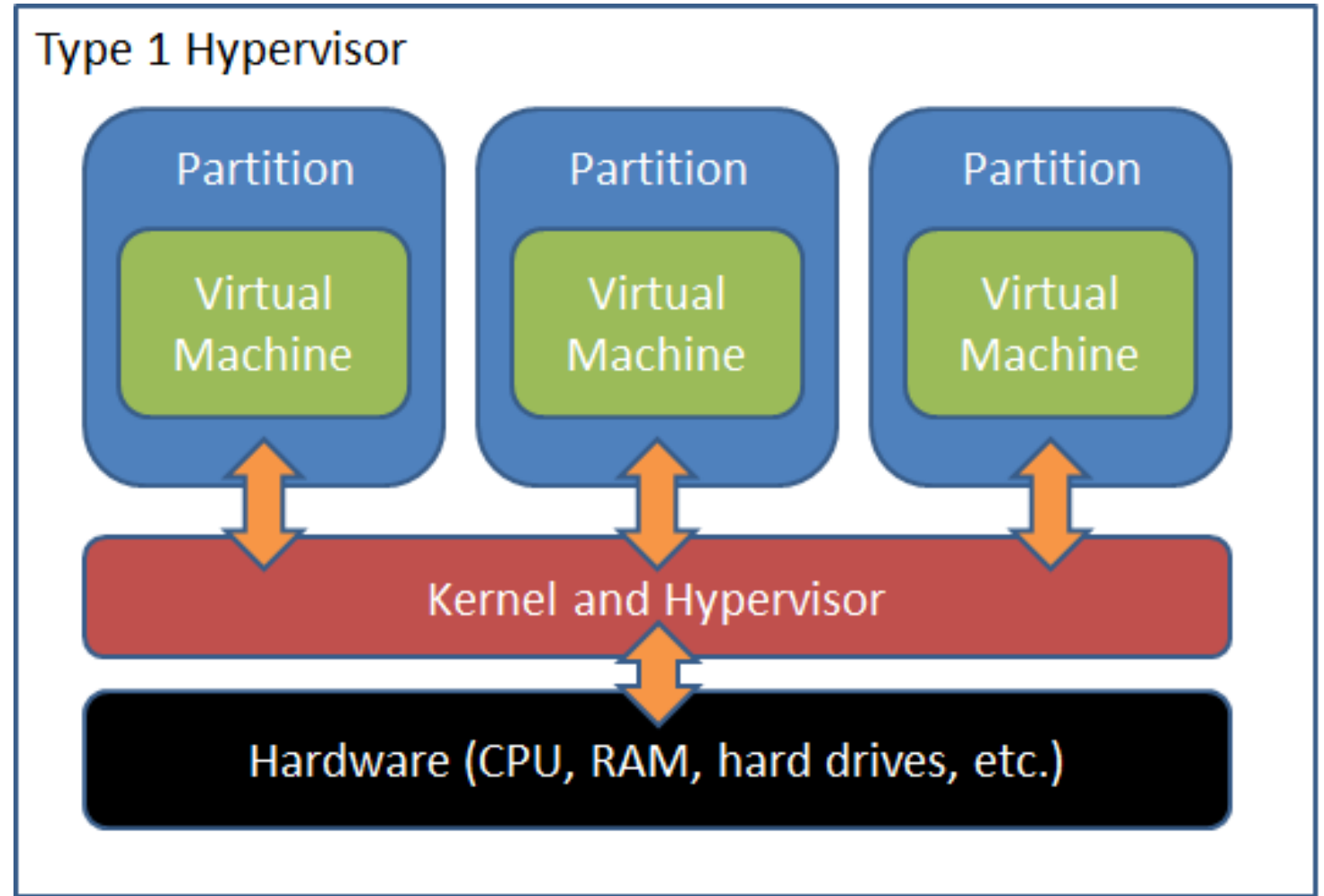


Type 1 Hypervisor



Máquina virtual: Balance

- Pros
 - Portabilidad
 - Seguridad (+-)
 - Máquina general más segura
 - Máquina virtualizada a riesgo del uso
- Contrás
 - *Overhead*
 - Performance
 - Redundancia
 - Eficiencia
 - Costo





Estilos y Patrones II

IIC2173 - Arquitectura de sistemas de software

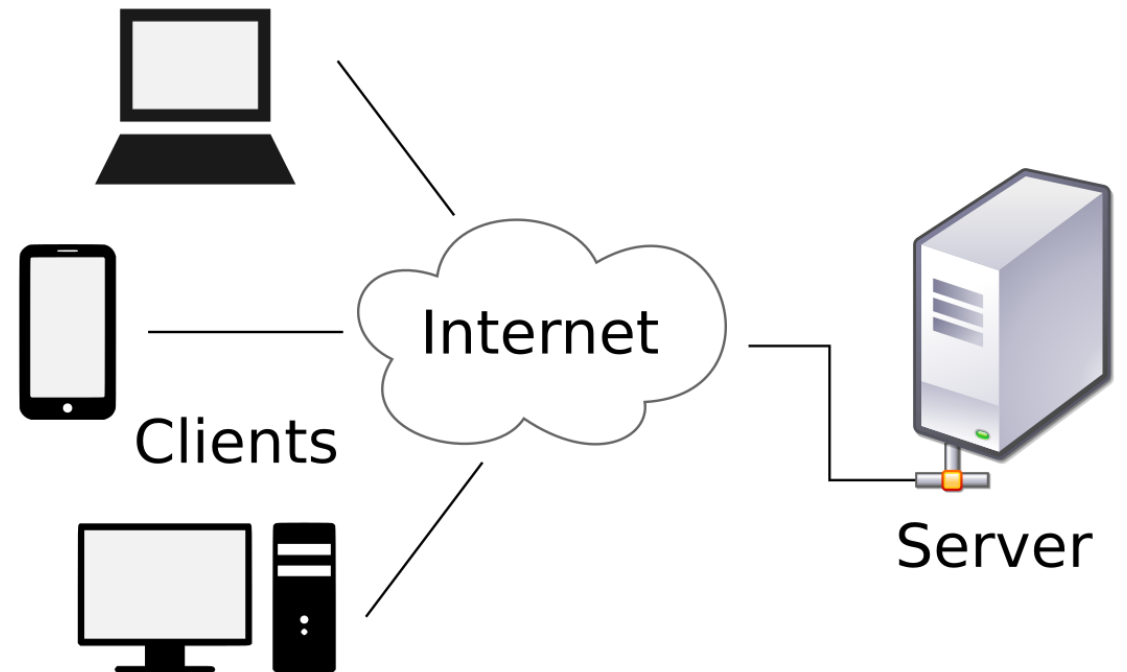
Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

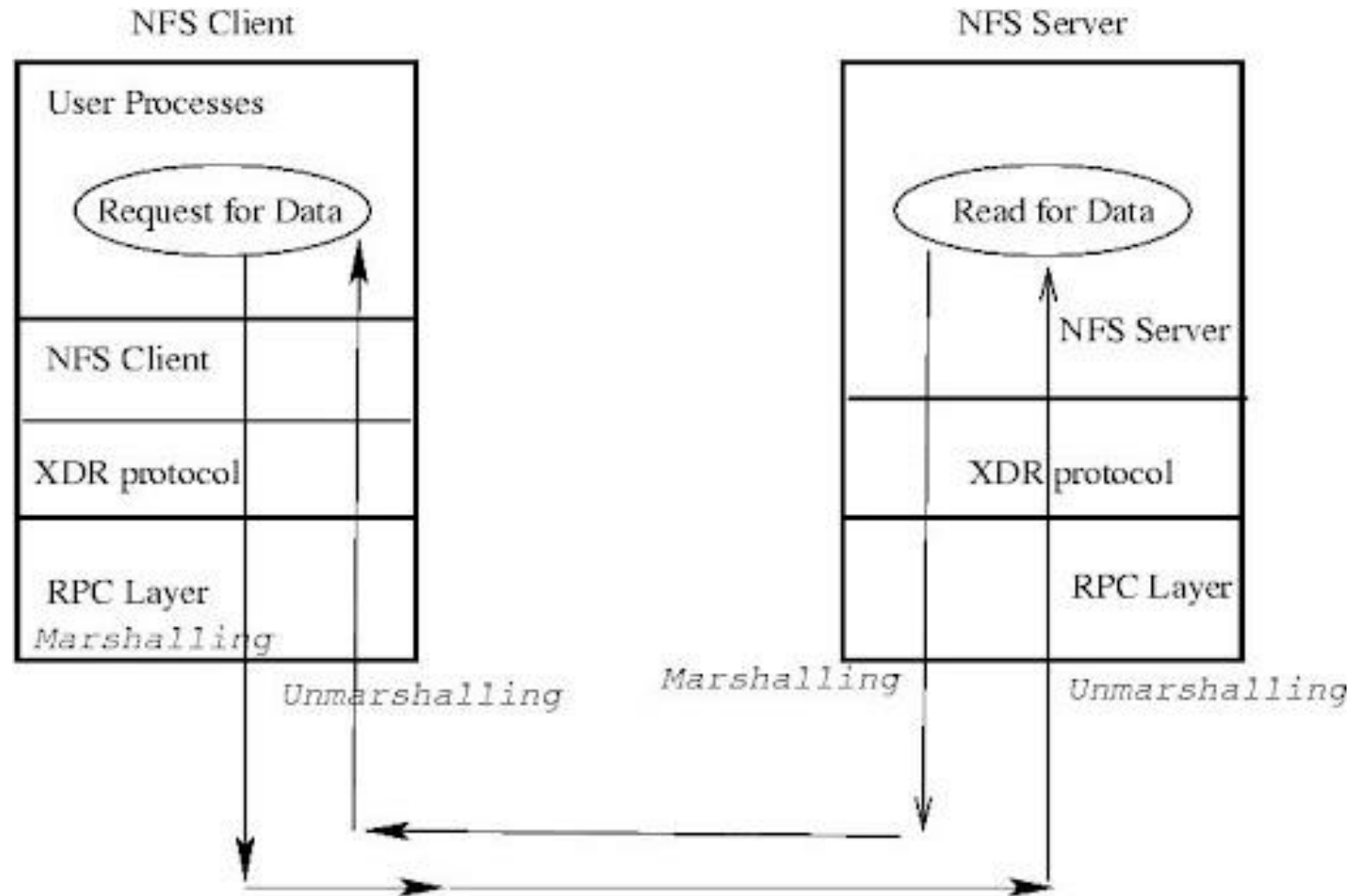
Variante capas: Cliente - Servidor

- División de capas en dos categorías
 - Cliente
 - Servidor
- Muy común en la actualidad
- Conocimiento asimétrico
 - Servidores no conocen a clientes
 - A priori: posteriormente puede usar identificadores
 - Clientes conocen al servidor
 - Relación 1-N



Cliente – Servidor: Comunicación

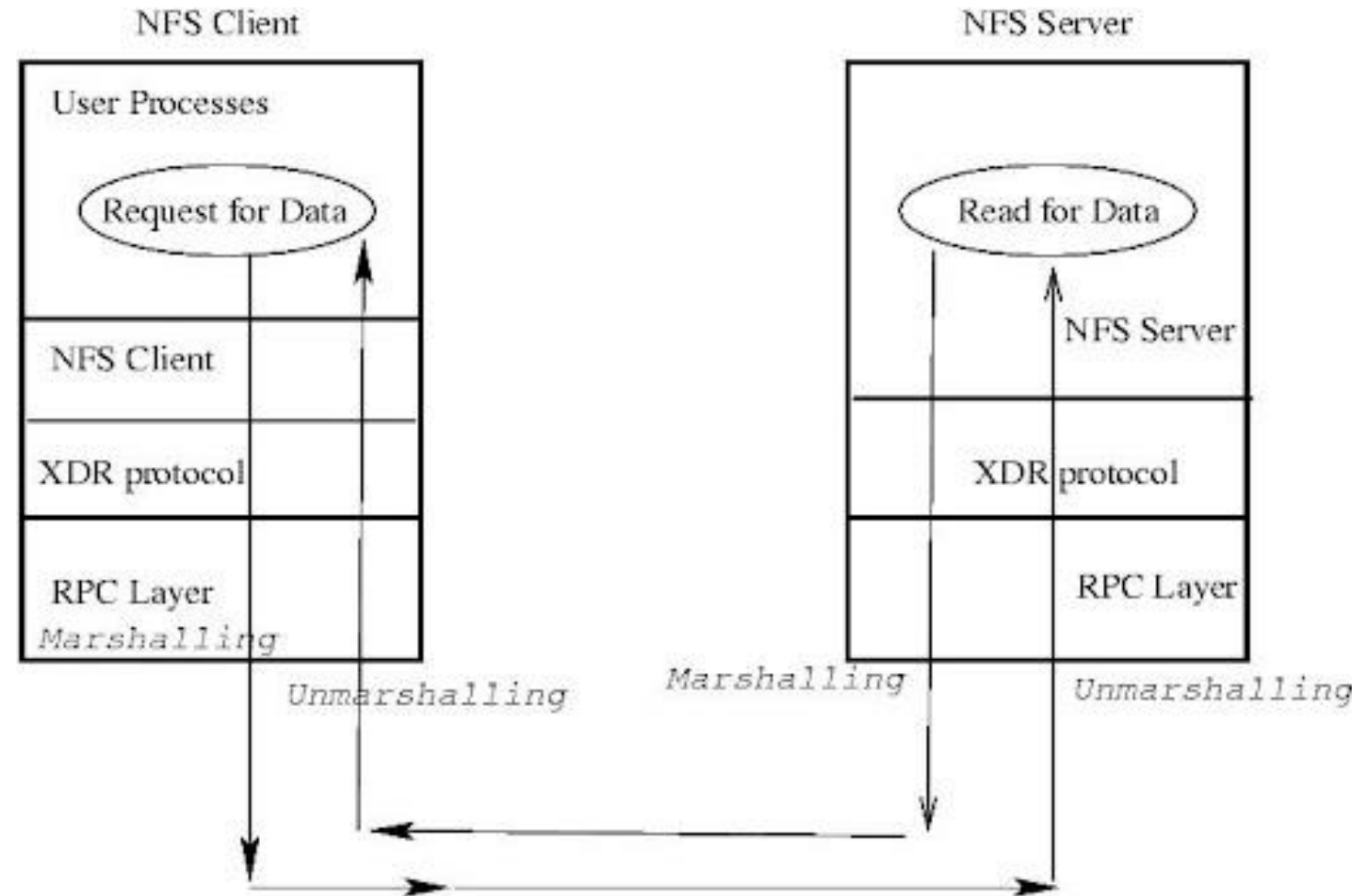
- REST (lo veremos más adelante)
- RPC - *Remote Procedure Call* (RPI - ...*Invocation*)
 - Cliente invoca a un método o procedimiento que se ejecuta remotamente simulando ser local
 - Espera la respuesta
 - *Off-loading* de procesamiento
 - Posibilidad de *thin-client*



Cliente – Servidor: Comunicación

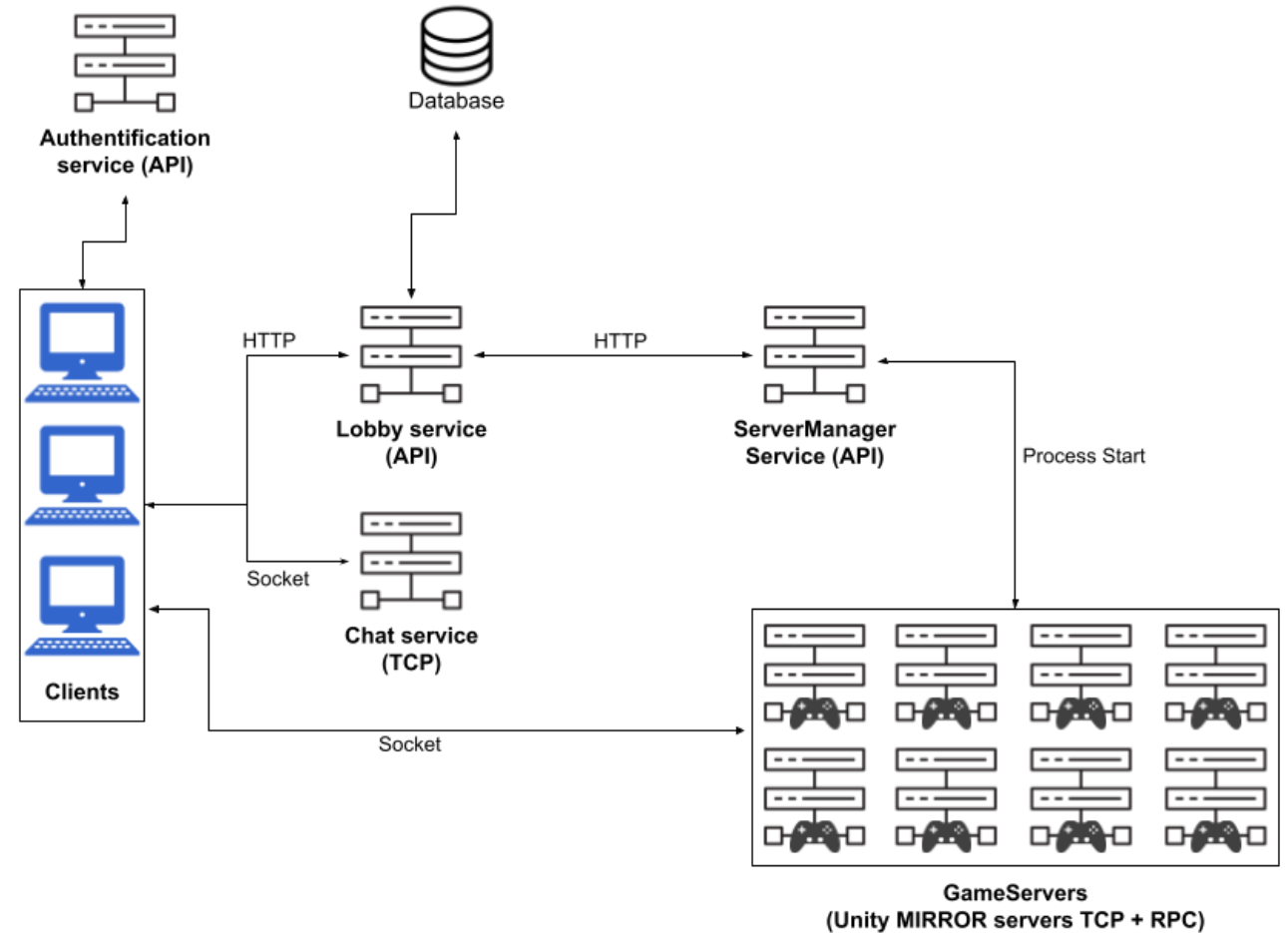
Cliente debe saber cómo hacer la llamada

- Conoce la interfaz
- Sabe cómo empaquetar y desempaquetar para enviarlo a la red
 - *Marshalling & Unmarshalling*
- Conoce la dirección del servidor



Cliente servidor: Ejemplo

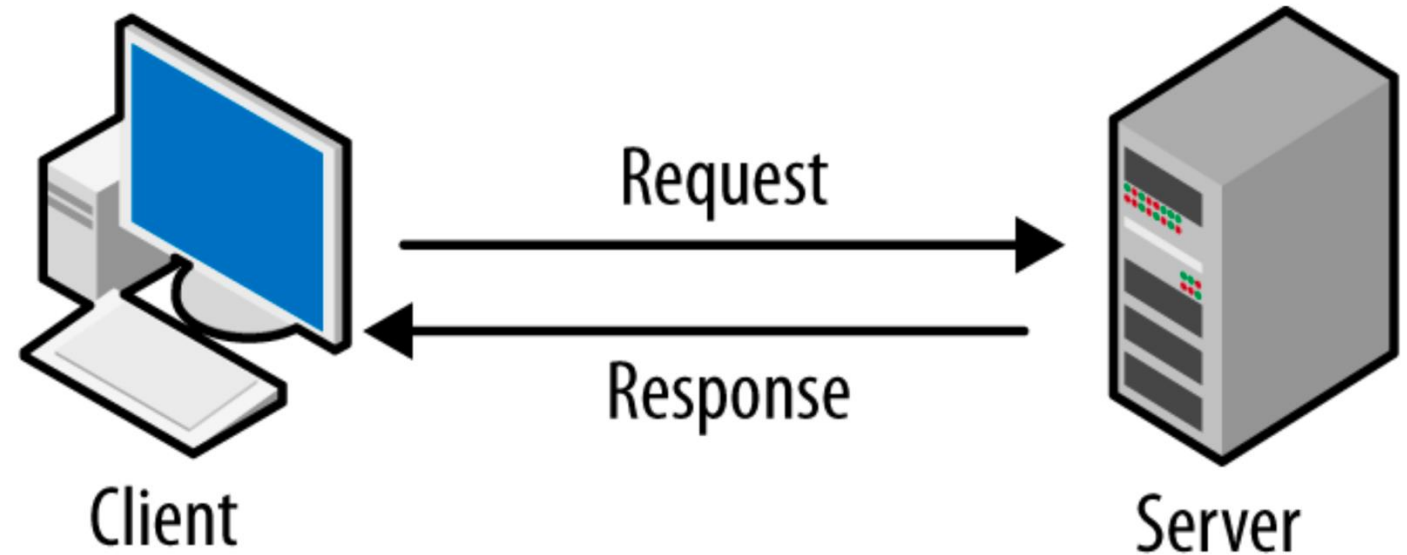
- Arquitectura web
- SMTP
- APIs web
- Servidores gaming
- Video streaming



Unity

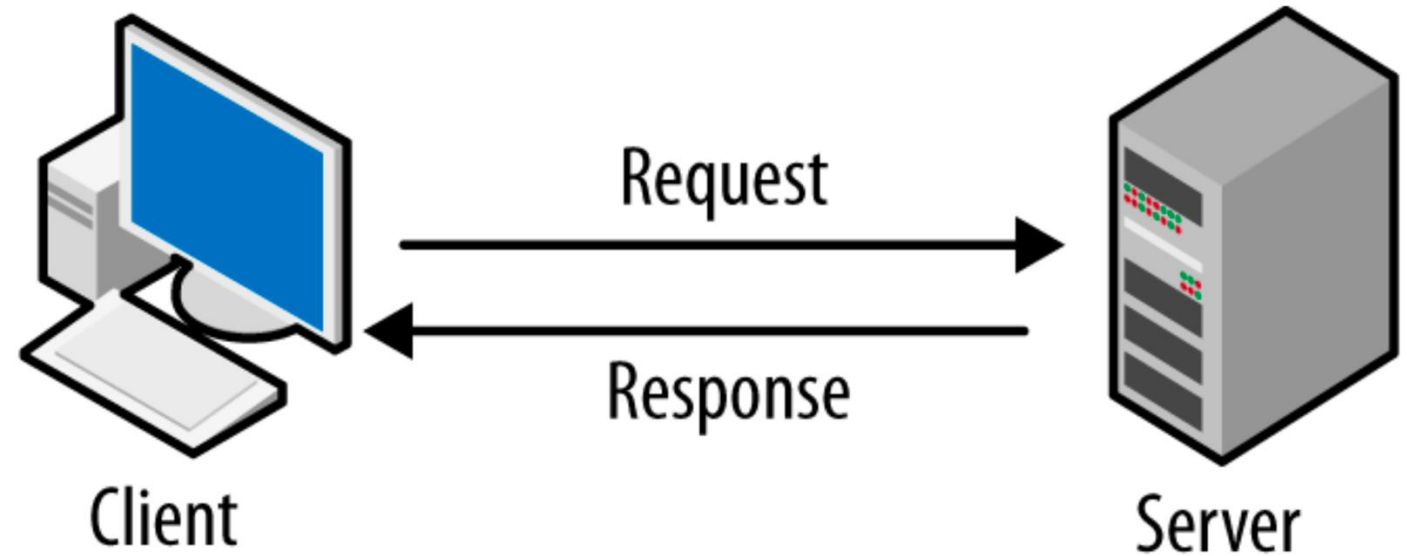
Ciente servidor: Pros

- Confiable
 - Apto para introducir mecanismos de confiabilidad
- Sencillo y probado
- Altamente mantenible



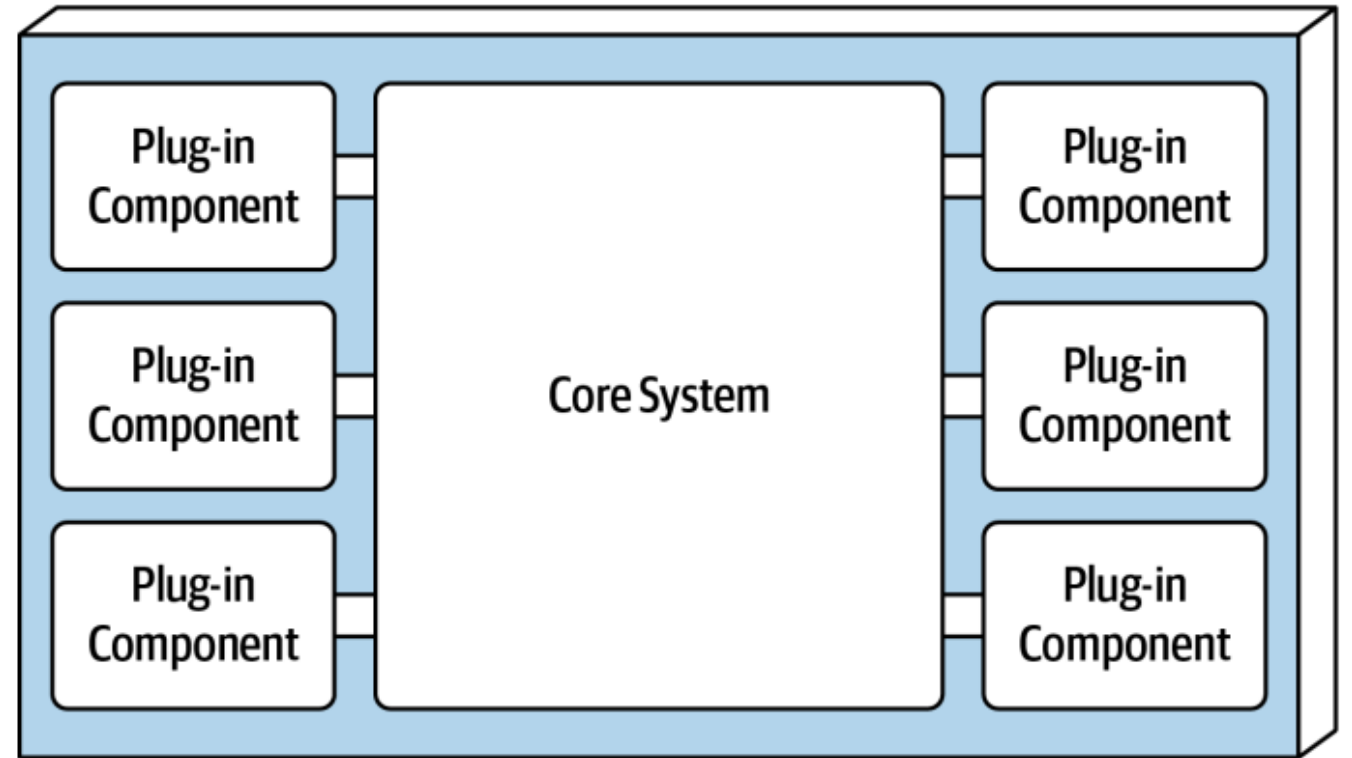
Cliente servidor: Cons

- Ojo con centralización
 - Único punto de fallo en su forma básica
 - Muy pesado en términos de red
- Puede crecer en tamaño fácilmente si no se controla



Microkernel

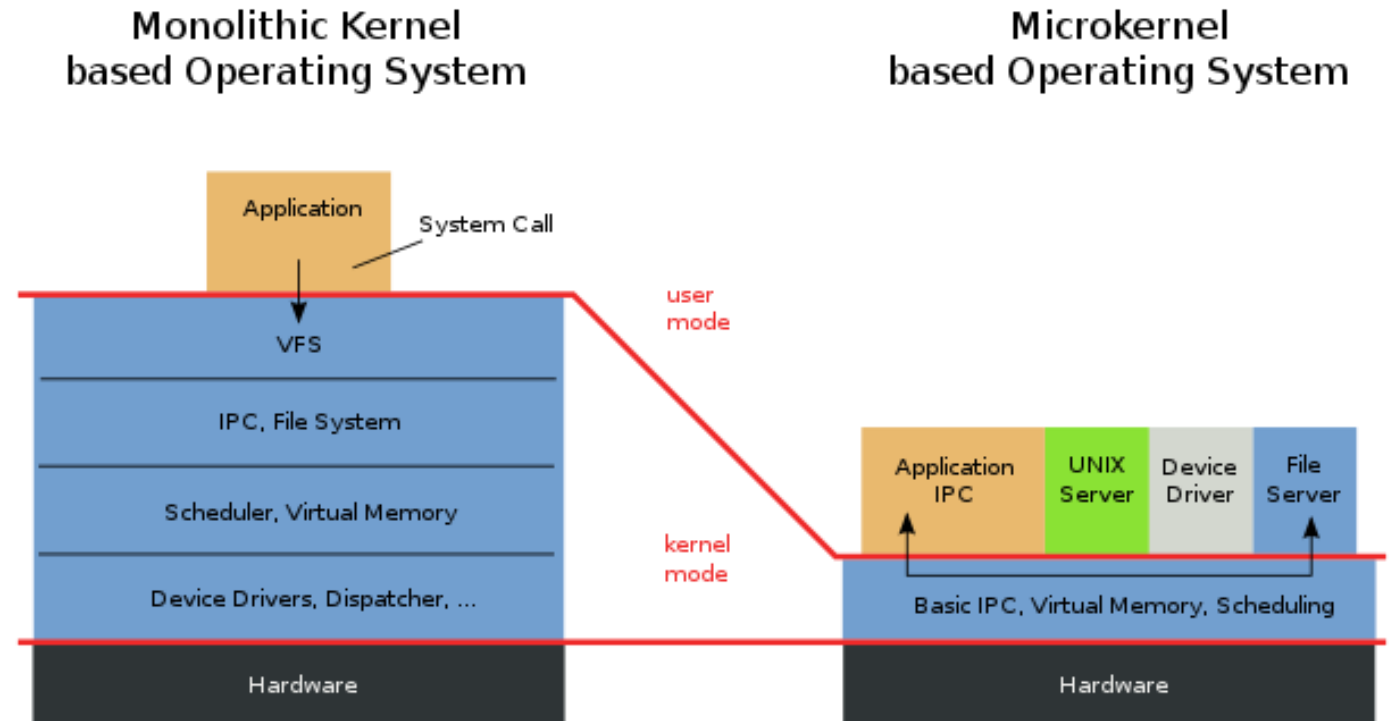
- Funcionalidad *core* mantenida al mínimo posible
- Se añaden otros sistemas vía *plugins* con interfaces estandarizadas para extender el sistema
- Comunicación generalmente punto a punto
- Usualmente asociado con *kernels*, pero actualmente se acerca a muchas arquitecturas



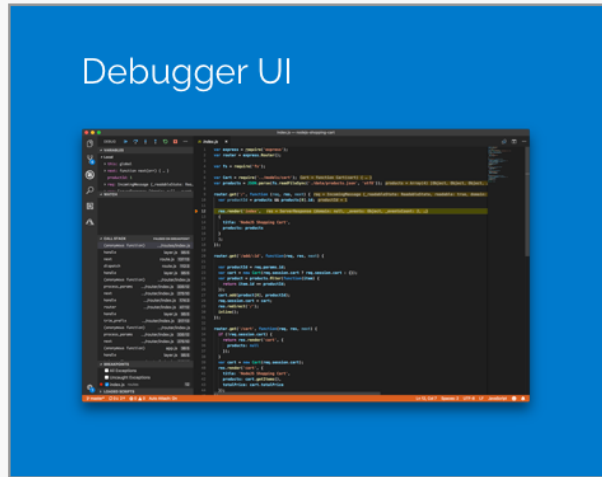
FOSA

Microkernel: Ejemplos

- Kernels...
 - Apreciado por extensibilidad
- IDEs
- Navegadores
- Sistemas multi-entorno



VS Code



Debugger UI

→ **Debug
Adapter
Protocol**

VS Code Extensions

Node.js
**Debug
Adapter**

Python
**Debug
Adapter**

C#
**Debug
Adapter**

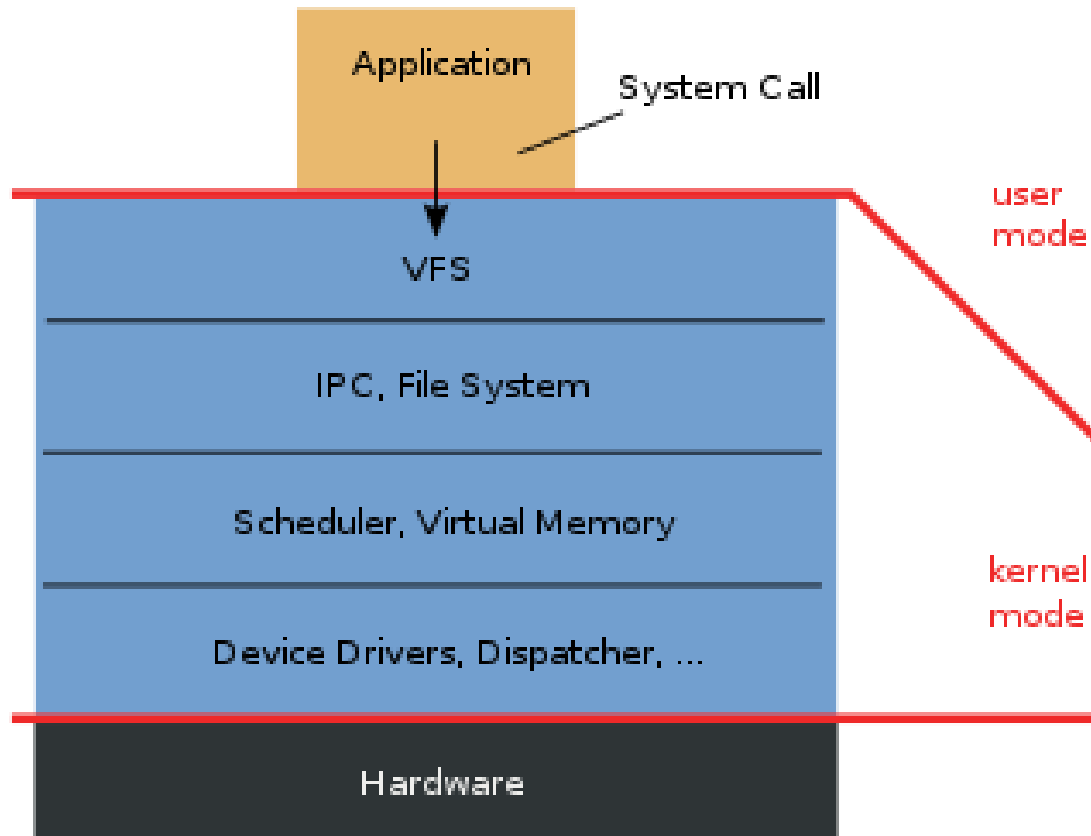
Runtimes

Node.js

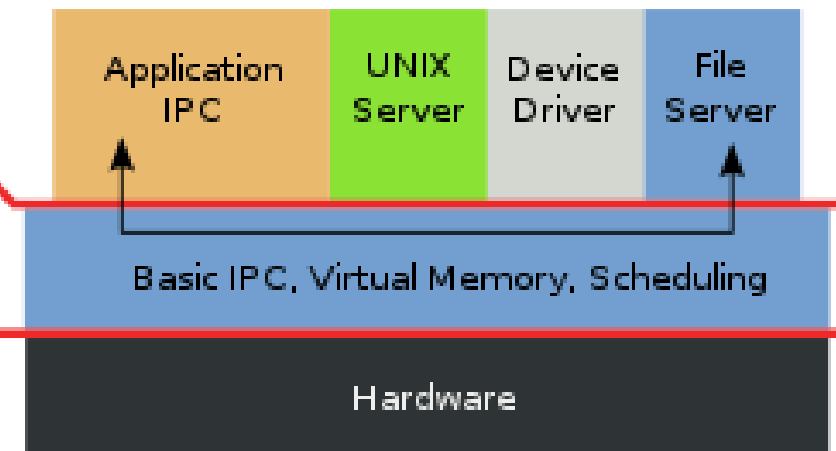
Python

C#

Monolithic Kernel based Operating System

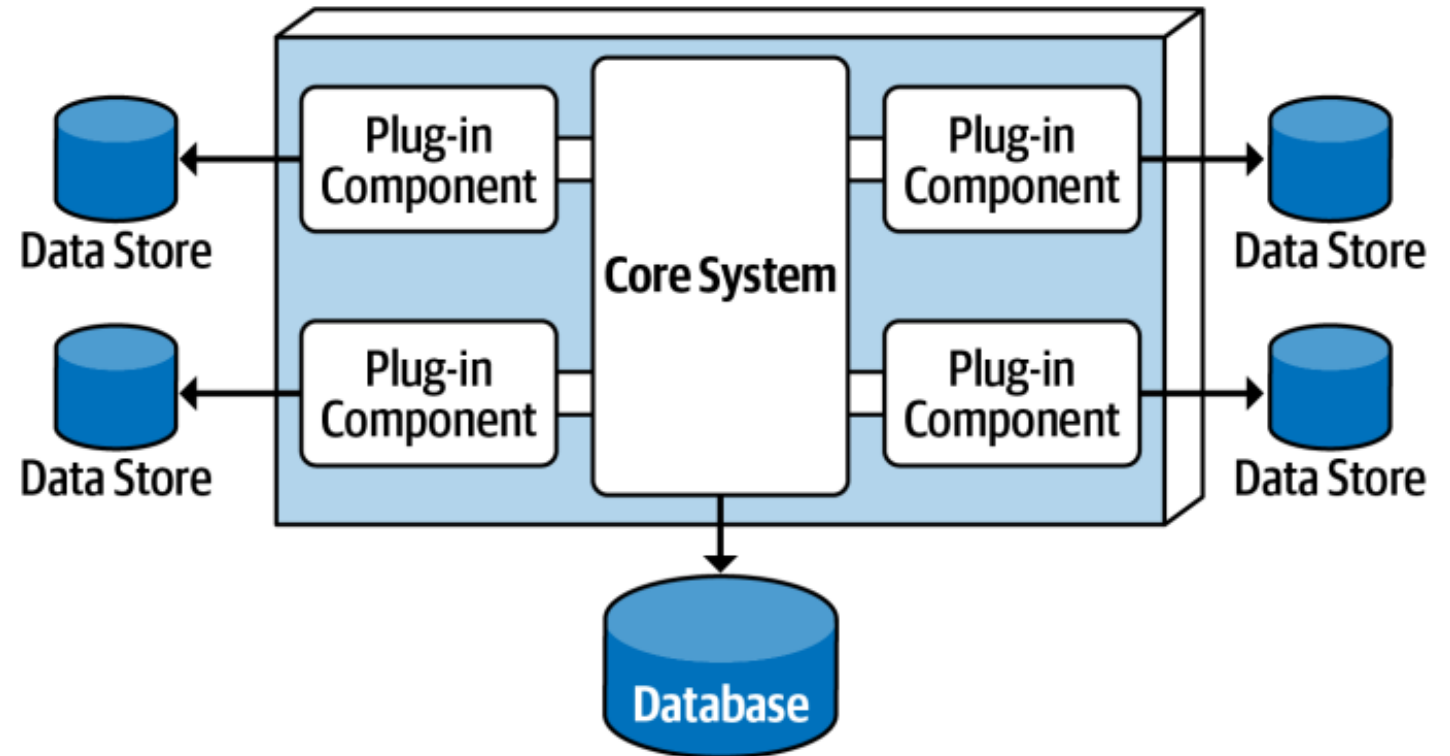


Microkernel based Operating System



Microkernel: Variantes y topologías

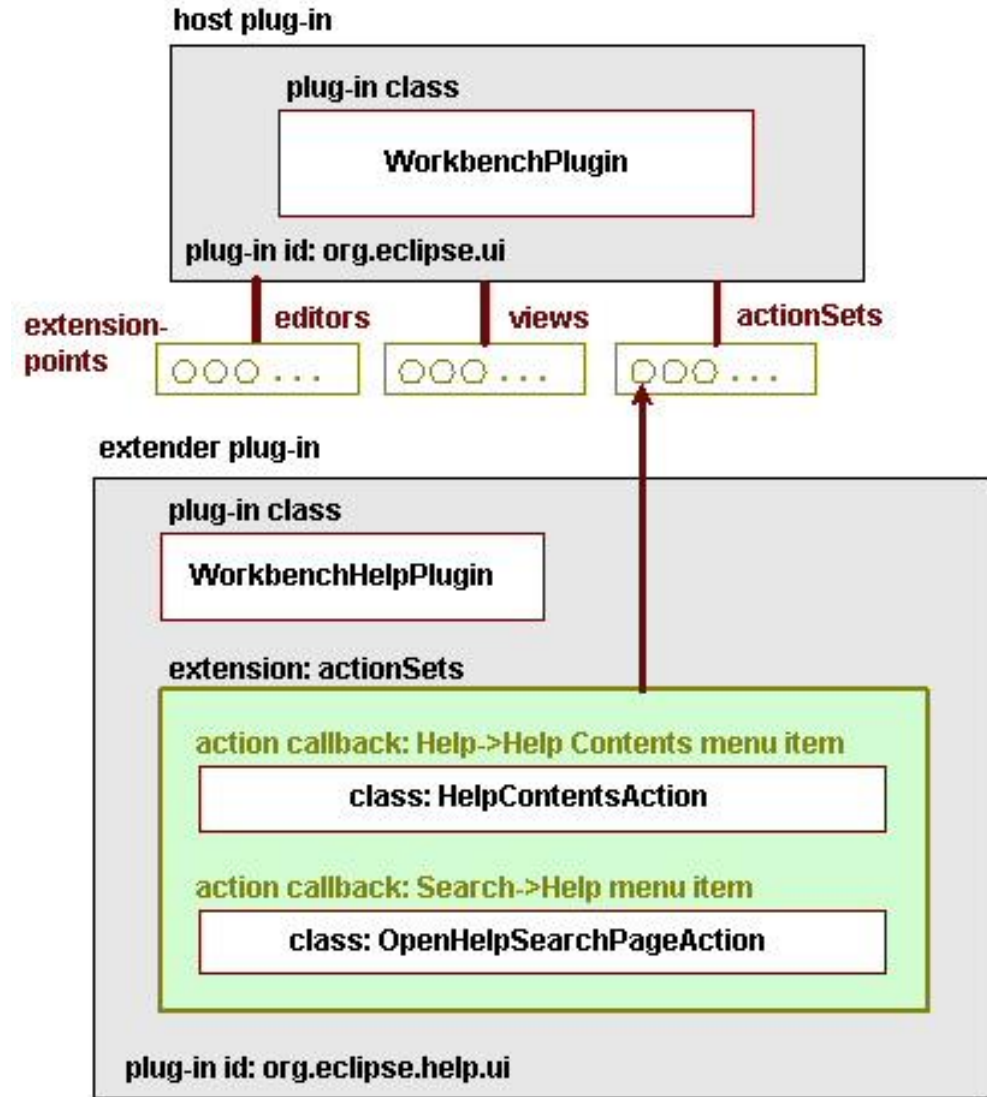
- Los enlaces pueden no solo ser *calls* de funciones
 - REST
 - Queues
- Se pueden agregar *datastores*
 - Común
 - Por plugin
 - Al sistema core
- Puede tener un registro de *plugins* para buscarlos fácilmente



FOSA

Microkernel: Pros

- Fácil de evolucionar
 - Miles de extensiones en marketplaces
- Fácil de implementar
 - Simple, económico
- Incorpora particionado de dominio
 - Puede reflejar mejor ciertos problemas



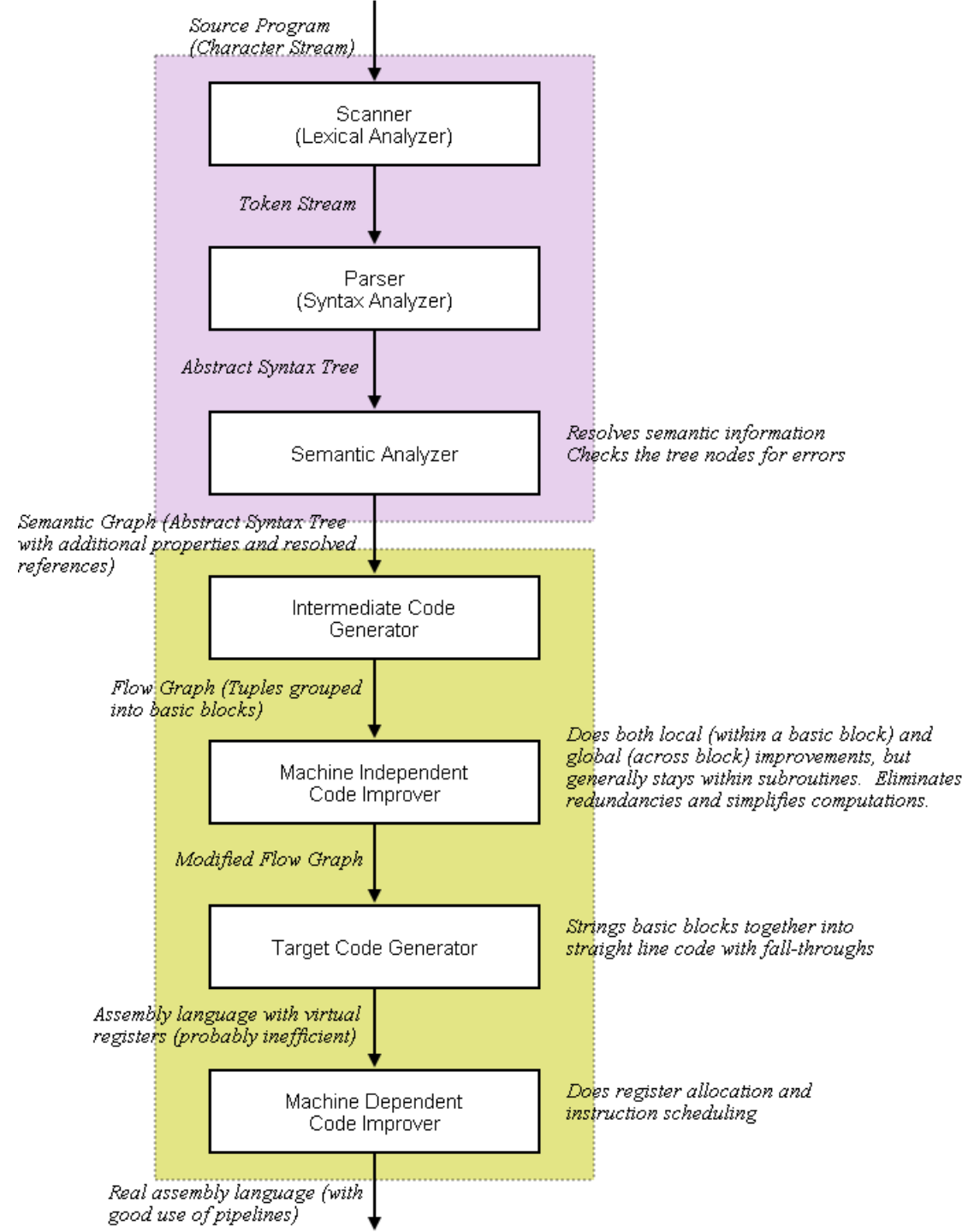
Microkernel: Cons

- Punto único de falla
- Quanta única
- Mala escalabilidad
 - Limitado por el *core*
- Puede responder mal a la demanda



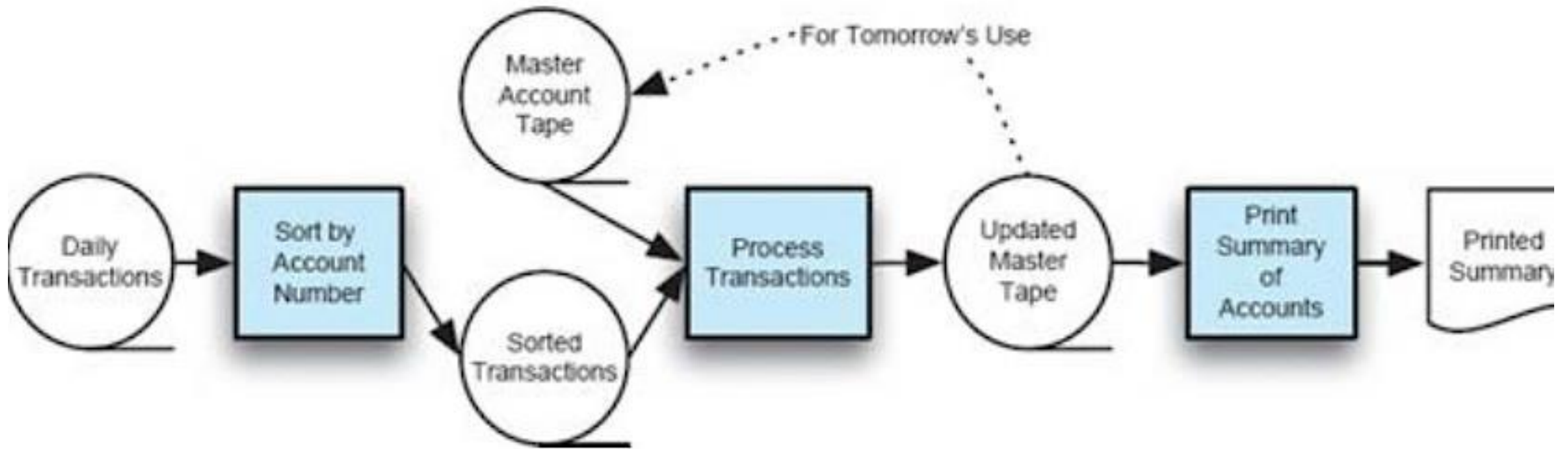
Flujo de datos: Por lotes, secuencial

- Programas separados **se ejecutan en orden**
- Aplicaciones: por lotes
 - Sistemas de banca
 - Sistemas de inventario
- Aplicaciones: Secuencial
 - Control y robótica
 - Compiladores



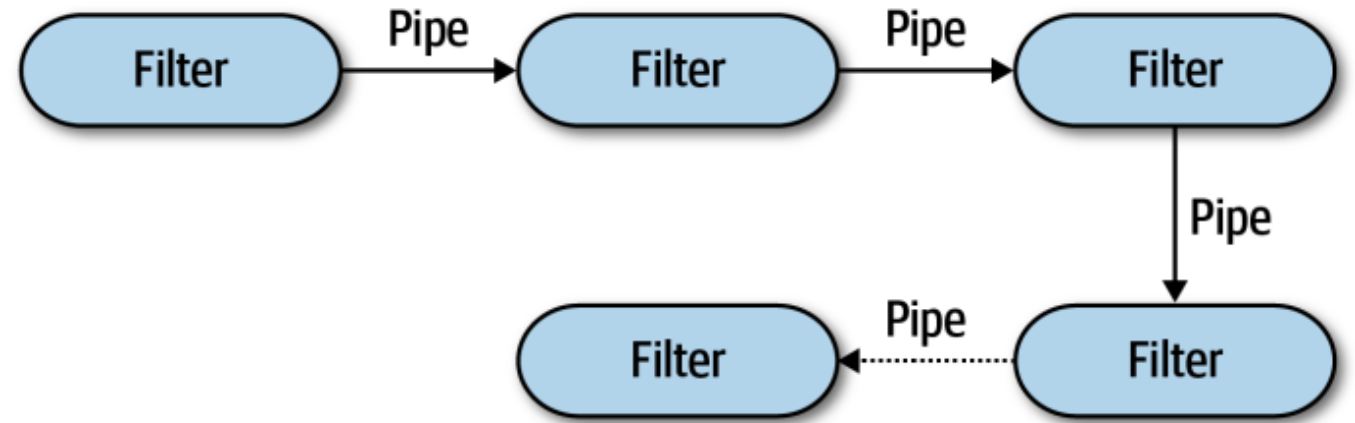
Por lotes, secuencial

- Un solo programa ejecutándose
- Topología lineal: datos van paso a paso
- Pequeños programas dependientes unos de otros
- Sencillo, asociable a otros sistemas



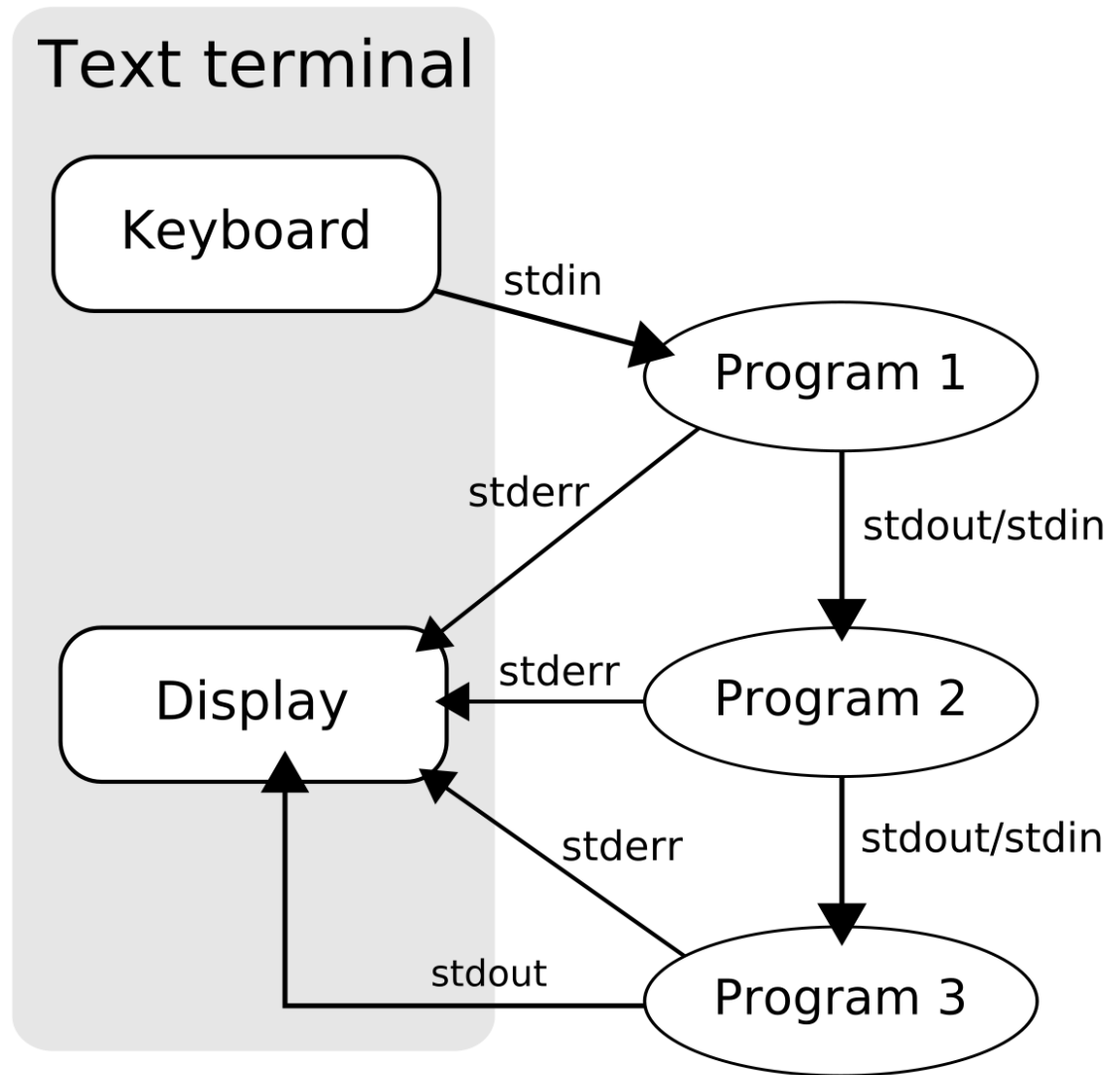
Flujo de datos: Pipe and Filter

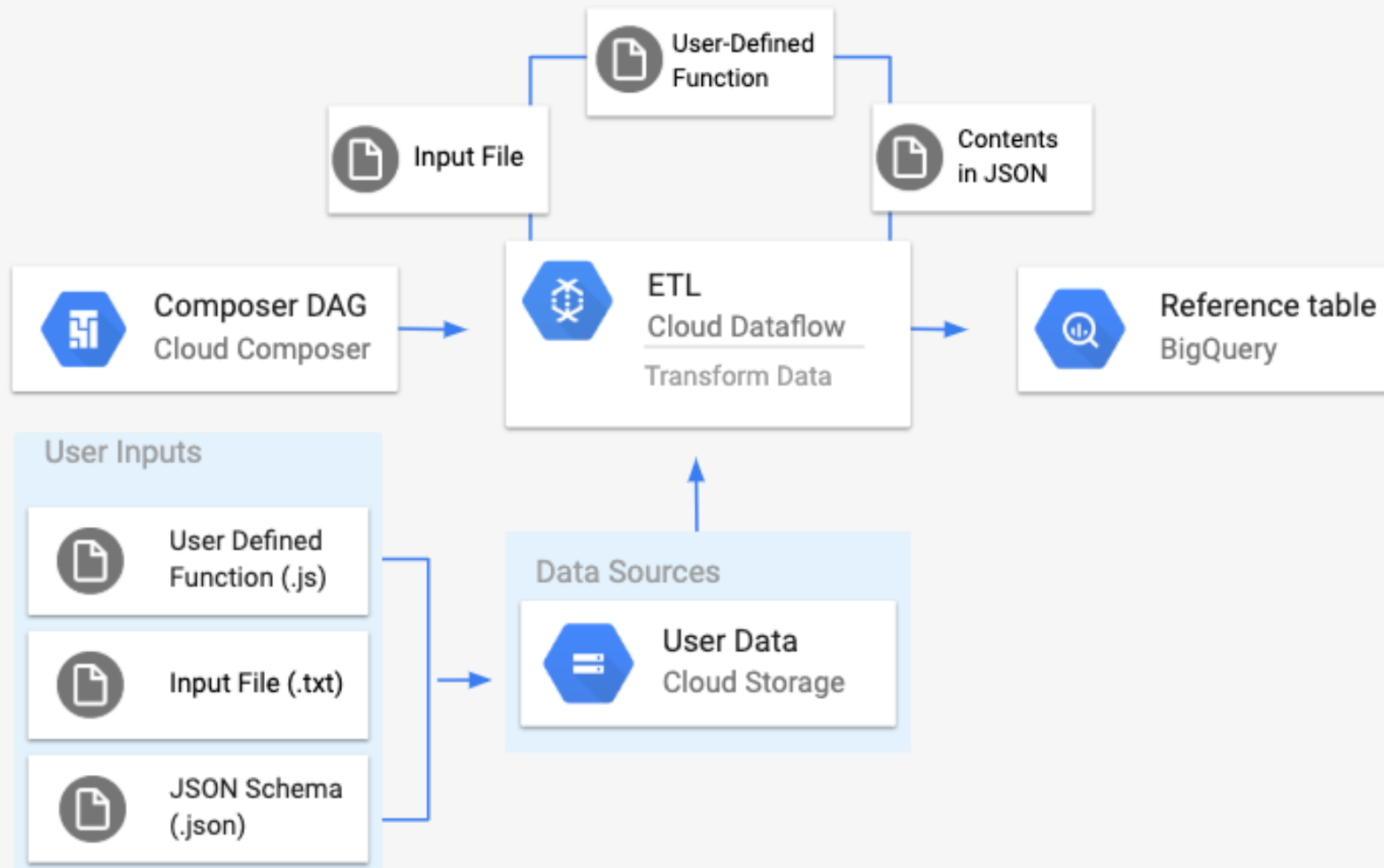
- Estilo sumamente poderoso
- Componentes con **interfaces claras**
- Típicamente los pipes son unidireccionales y de punto a punto
- Cada componente Filtro **cumple una función encapsulada**

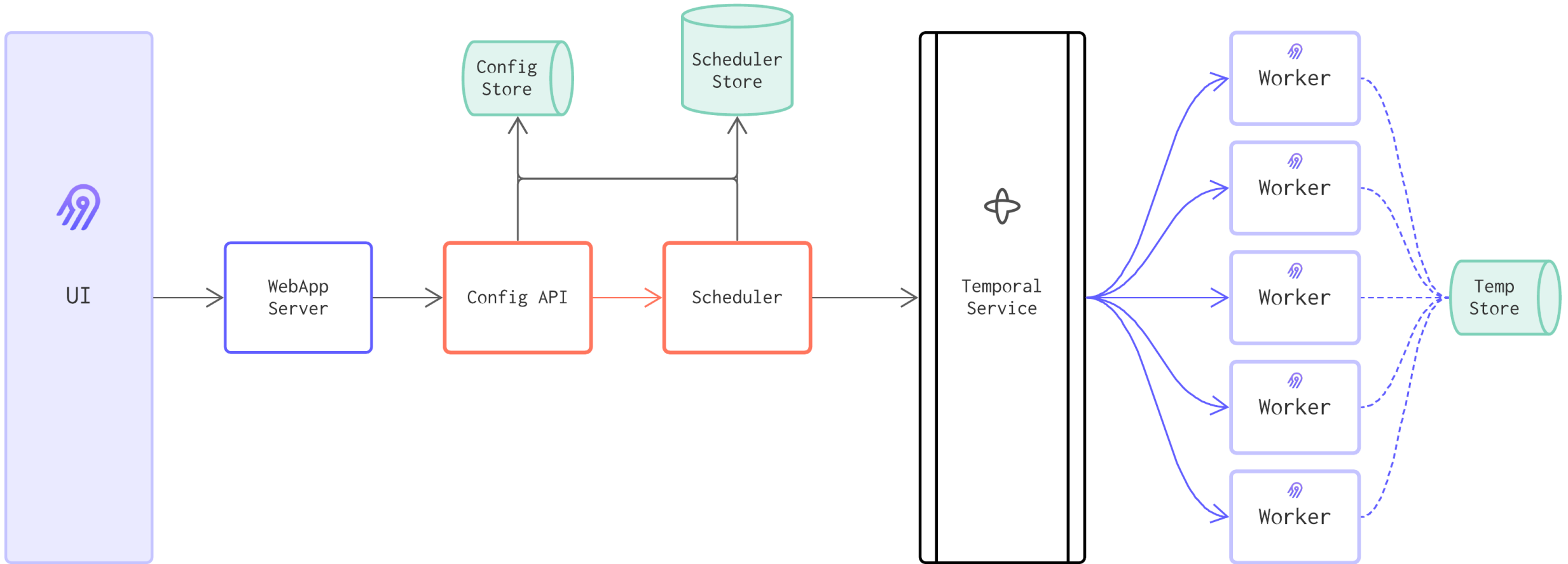


Pipe and Filter: Ejemplos

- Pipes en Linux
- Pipes de procesamiento de objetos (e.g. machine learning en imagen)
- Procesamiento de audio
- ETLs

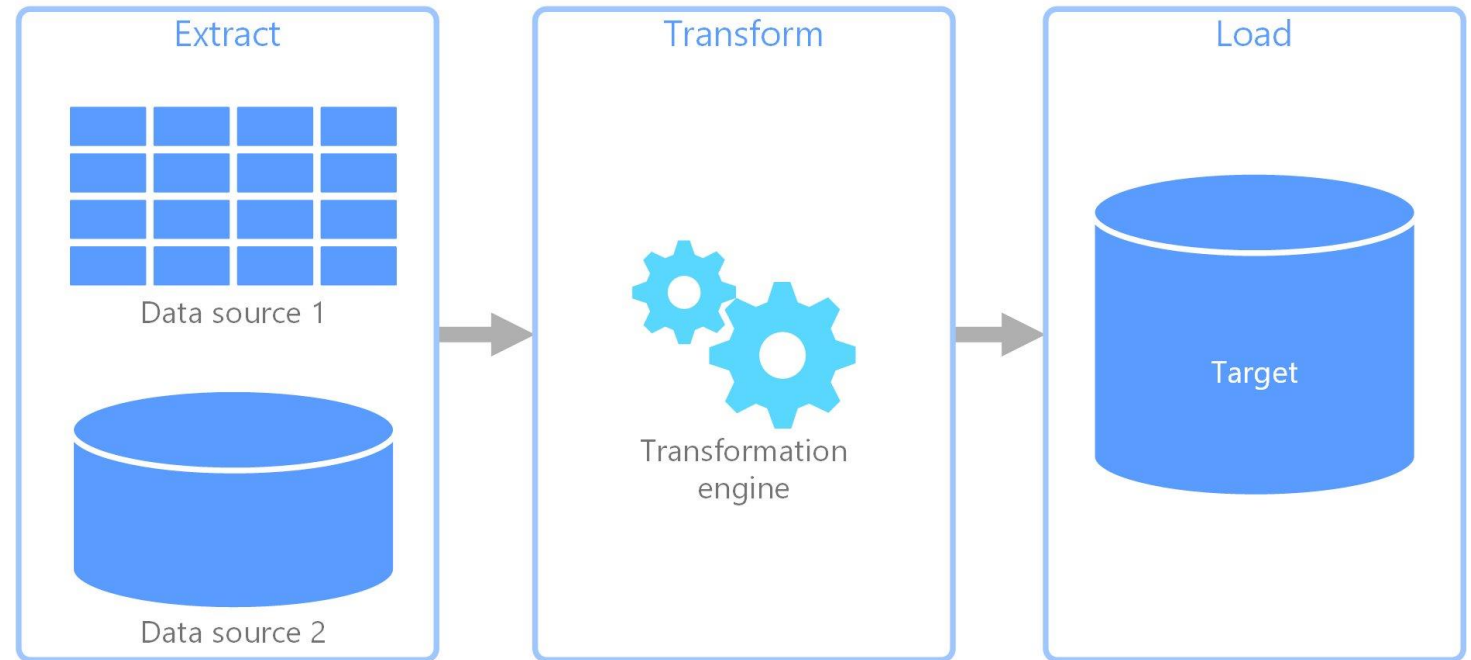






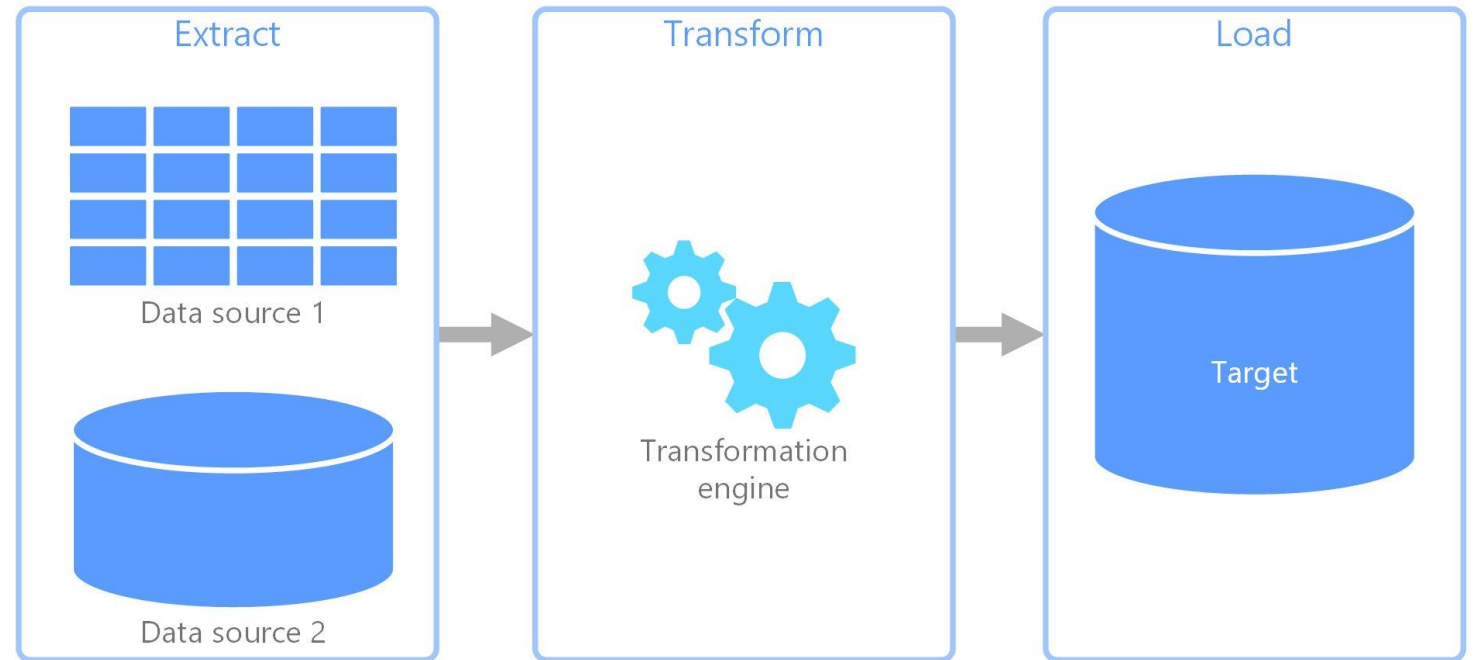
Pipe and Filter: Pros

- Modular
 - Fácil de modificar módulos y mantener
- Reusable
- Cada componente puede ser escalable
- Flexible
- Testeabilidad



Pipe and Filter: Cons

- Requiere un **proceso organizado**
 - No soporta aplicaciones interactivas
- Complejidad
- A medida que el sistema se extiende, **se acopla más**
- No toda la data es compatible
- Se puede acumular *overhead*
- Mínimo común denominador en la transmisión de datos





Estilos y Patrones II

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot